目录

0x00 基本计数原理 3
0x01 加法、乘法、减法、除法原理 3
0x02 抽屉原理（鸽巢原理） 6
0x03 容斥原理 9
0x10 排列 14
0x11 排列 14
0x12 全排列 15
0x13 错位排列与圆排列 16
0x20 组合 16
0x21 组合 16
0x22 组合数的性质 17
0x23 求组合数 17
0x23.1 加法递推 O(n^2) 18
0x23.2 乘法递推 O(n) 18
0x23.3 Lucas定理 19
0x23.4 扩展卢卡斯 21
0x23.5 求大组合数（高精） 23
0x23.6 约分求重数 25
0x24 计数技巧 26
0x24.1 捆绑法 26
0x24.2 插空法 26
0x24.3 隔板法 27
0x30 多重集 31
0x31 多重集的排列数问题 31
0x32 多重集的组合数问题 31
0x40 二项式定理 32
0x41 一般二项式定理 32
0x42 扩展二项式定理 32
0x43 广义二项式定理 32
0x50 特殊的数列 32
0x51 斐波那契数（\text{Fibonacci}） 32
0x52 卡特兰数（\text{Catalan}） 33
0x52.1 卡特兰数 33
0x52.2 卡特兰数的应用 33
0x52.3 求卡特兰数 34
0x52.4 竞赛例题选讲 35
0x53 斯特林数（\text {Stirling}） 36
0x53.0 上升幂与下降幂 36
0x53.1 第一类斯特林数 37
0x53.2 第二类斯特林数 43
0x53.3 第三类斯特林数 45
0x53.4 三类斯特林数之间的转换 47
0x52.5 斯特林反演 47
0x54 贝尔数（\text{Bell}） 48
0x54.1 贝尔数 48
0x54.2 \text{Bell} 三角 48
0x55 分拆数 48
0x56 伯努利数（\text {Bernoulli}） 48
0x60 生成函数 49
0x61 普通生成函数 49
0x62 指数生成函数 50
0x70 群论 50
0x71 群 50
0x71 Burnside引理与Polya定理 51
0x22 Polya 定理 52
0x23 特殊模型 53
0x23.1 旋转同构 53

0x23.2 对称同构 53
0x80 递推方程.....54
0x90 康托展开.....54
0x91 康托展开 54
0x92 逆康托展开.....55

0x00 基本计数原理

0x01 加法、乘法、减法、除法原理

▲ 加法原理（分类）

定义： 若完成一件事有 n 类方式，第 i 类有 a_i 种方法，那么完成这件事共有 $a_1+a_2+\cdots+a_n$ 种方法。

经典应用： 从北京到上海有火车、飞机、轮船 3 种方式，火车、飞机、轮船分别有 $k1$，$k2$，$k3$ 个班次，那么从北京到上海有 $k1+k2+k3$ 种方 式可以到达。

▲ 乘法原理（分步）

定义： 若完成一件事n个步骤，第i个步骤有a_i种方法，那么完成这件事共有$a_1\times a_2\times\cdots\times a_n$种方法。

经典应用： 从北京乘坐火车到上海，需要转 3 次车，每次专车分别有 $k1$，$k2$，$k3$ 个班次，那么从北京到上海有 $k1\times k2\times k3$ 种方式可以到达。

▲ 减法原理（正难则反）

定义： 记 S 为全集，则 $|A|=|S|-|S\setminus A|$，常用于当直接统计具有一种性质的事物个数较为困难时，考虑统计**不具有这种性质的事物个数**，再用总数减去这个值。

经典应用： 班上一共有 n 个同学，老师点名，到了 m 个，则有 $n-m$ 个同学没来。

▲ 除法原理（划分）

定义： 令 S 是一个有限集合，把它划分乘 k 个部分使得每一部分包含的对象数目相同，于是。次划分中的部分的数目由下述公式给出：$k=\dfrac{|S|}{在一个部分中的对象数目}$

因此，如果我们知道 S 中的对象数目，以及各部分所含对象数目的共同之， 就可以确定部分的数目。

Hint

加法原理和乘法原理是两个计数基本原理，回答的都是有关做一件事的不同方法种数的问题。

二者区别在于：

分类加法原理针对的是分类问题，其中各种方法**相互独立**，用任何一种方法都可以完成这件事。

分步乘法原理针对的是分步问题，各步骤中的方法相互依存，只有各个步骤**都完成**才算完成任务。

分类要依据同一标准划分，既必须**包括所有情况**，又不要交错在一起产生重复。

分步则应使**各步依次完成**，保证整个任务得以完成，既不得多余重复，也不缺少某一步骤。

减法原理则是一个常用的计数技巧，**正难则反**。

▲ 竞赛例题选讲

Problem A Lining Up（AtCoder 2271）

有 n 个人站一行，对于第 i 个人，给出第 i 个人站在其两端的人数的差值 `a[i]`，问一共有多少种可能的站法。若不存在这样的排列方法，输出 `0`。

Solution

由于我们仅有的信息是 n 的值，和 a_i 所以我们考虑对 n 进行讨论。

对 n 进行奇偶讨论：

- 当 n 为奇数时：中间的人的左右的人的个数一定相同，因此 a_i 为 0 且只有 1 个，从其向两边延伸，每延伸 1 个人，差的人数 a_i+2，因此在奇数情况下，只有 0、2、4、6、8、$\ldots$。也就是，除 0 有 1 个外，其余均为 2 个人。
- 当 n 为偶数时：一定没有两边差值 a_i 为 0 的情况，其两边的差最小为 1，从其两边延伸，每延伸 1 个人，差的人数 a_i+2，因此在偶数的情况下，只有 1、3、5、$7\ldots$，均为 2 个。

综上所述，除了奇数情况时差值 a_i 为 0 的在中间外，其余的每个位置有两个人可选（就是 a_i 相同的两个人位置可以交换），因此根据乘法原理，答案为 $2^{\lfloor \frac{n}{2} \rfloor}$ 。

至于无解的情况，根据上述分析，我们可以开一个桶，记录每个数的个数，如果不满足上面分析的 奇数情况的 0、2、4、6、8、$\ldots$ 除 0 有 1 个外，其余均为 2 个人，或者偶数情况的 1、3、5、$7\ldots$，且均为 2 个即为无解输出 `0`。

Code

```cpp
typedef long long ll;
typedef int itn;
const int N = 500007, M = 5000007, INF = 0x3f3f3f3f, mod = 1e9 + 7;
const ll LINF = 4e18 +7;
int n, m;
ll a[N];
int vis[N];
int qpow(int a, int b);
int main()
{
    scanf("%d", &n);
    for(int i = 1; i ≤ n; ++ i) {
        scanf("%d", &a[i]);
        vis[a[i]] ++ ;
    }
    bool flag = true;
    if(n & 1) {
        if(vis[0] == 0)
            flag = false;
        for(int i = 2; i ≤ n; i += 2) //奇数一定是 2,4,6,8 ... 且一定都是两个才对称
            if(vis[i] ≠ 2) {
                flag = false;
                break;
            }
    }
    else {
        for(int i = 1; i ≤ n - 1; i += 2) //偶数一定是 1,3,5,7 ... 且一定都是两个才对称
            if(vis[i] ≠ 2) {
                flag = 0;
                break;
            }
    }
    ll res = qpow(2, n / 2);
    if(flag)
        printf("%lld\n", res);
    else puts("0");
    return 0;
}
```

Problem B Triangle Counting（UVA11401 ）

给定 n 条边，长度分别为 $1,2,3,\ldots,n$ 。用其中三条边构成一个三角形，有多少种不同的方案？注意，一条边只能使用一次。

$n \leq 10^6$

Solution

计算能组成三角形的选择方案数，为了补充不漏的计数，我们可以考虑固定三角形中的最长边 x，讨论另外两条边 y, z 的情况。

发现我们有且只有一个能用的性质：三角形中，两边之和大于第三边。

假定我们已知 x ，则有：

$$y + z > x \tag{1}$$

对于当前的 x，考虑合法的 y 的范围，显然有：

$$x - z < y < x \tag{2}$$

尝试讨论：当 $y=1$ 时，显然 z 无解。当 $y=2$ 时，z 只有 1 个解：$x-1 \leq z < x \Rightarrow z = x-1$；当 $y=3$ 时 z 有 2 个解：$x-2, x-1$， $\cdots$，当 $y=x-1$，z 有 $x-2$ 个解：$2, 3, \cdots, x-1$。显然方案数呈等差数列，故总的解数为：

$$\frac{(x-1)(x-2)}{2} \tag{3}$$

现在是满足 $y+z>x$ 的方案数，但是题目中规定每条边只能只用一次，所以要去掉 $y=z$ 的情况：

$$\begin{aligned} & y = z, x - z < y < x \\ & \Rightarrow x - y < y \Rightarrow y > \frac{x}{2} \\ & \Rightarrow \frac{x}{2} < y < x \\ & \Rightarrow \frac{x}{2} + 1 \leq y \leq x - 1 \end{aligned} \tag{4}$$

则共有 $x-1-(\frac{x}{2}+1)+1=\frac{x+2}{2}$ 种情况。

题目规定必须是不同的三角形，即 $y=10, z=5$ 与 $z=5, y=10$ 是同种方案，所以最后答案需要再除以二。

综上所述，答案 $f(x)$ 表示最长边为 x 时，可以组成的不同的三角形的方案数：

$$f(x) = \frac{\frac{(x-1)(x-2)}{2} - \frac{x+2}{2}}{2} = \frac{(x-2)^2}{4} \tag{5}$$

那么对于数列 $1, 2, \cdots n$，总方案数显然为 $\sum\limits_{i=3}^{n} f(i)$ 。前缀和预处理即可。

Code

```cpp
// Problem: UVA11401 Triangle Counting
// Contest: Luogu
// URL: https://www.luogu.com.cn/problem/UVA11401
// Memory Limit: 0 MB
// Time Limit: 1000 ms
//
// Powered by CP Editor (https://cpeditor.org)

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 2e6 + 7;

```

```cpp
int n, m, s, t, k, ans, a[N];
ll f[N];

void prework(int x)
{
    f[1] = f[2] = 0;
    for (int i = 3; i ≤ x; ++ i) {
        f[i] = f[i - 1] + 1ll * (i - 2) * (i - 2) / 4;
    }
}

int main()
{
    prework(N - 5);
    while(scanf("%d", &n) ≠ EOF && n) {
        if(n < 3) return 0;
        cout << f[n] << endl;
    }
    return 0;
}
```

0x02 抽屉原理（鸽巢原理）

▲ 第一抽屉原理

定理2.1： 把 $n+1$ 件东西放入 n 个抽屉，则至少有一个抽屉里会放入**两件或两件以上**的东西。

定理2.2： 从另一个角度考虑，把 $n-1$ 件东西放入 n 个抽屉，则至少**一个抽屉是空的**。

定理2.3： 把多于 $m \times n+1$（n 不为 0）个的物体放到 n 个抽屉里，则至少有一个抽屉里有不少于 $m+1$ 的物体。

定理2.4： 把比无数多还多的这么多件物体放入 n 个抽屉，则至少有一个抽屉里有无数个物体。

▲ 第二抽屉原理

定理2.5： 把 $m \times n-1$ 个物体放入 n 个抽屉中，其中必有一个抽屉中至多有 $m-1$ 个物体。

定理2.6： 把多于 $m \times n$ 个的物体放到 n 个抽屉里，则至少有一个抽屉里有不少于 $m+1$ 的物体。

例如：将 $3 \times 5-1=14$ 个物体放入 5 个抽屉中，则必定有一个抽屉中的物体数少于等于 $3-1=2$ （平均 3 个都不够，令一个变多，另一个一定变少）。

推论2.7： 已知 $n+1$ 个互不相同的正整数，它们全都小于或等于 $2n$ ，证明当中一定有两个数是互质的。

考虑证明：

取 n 个盒子，在第一个盒子我们放 1 和 2，在第二个盒子我们放 3 和 4 ，第三个盒子是放 5 和 6 ，依此类推直到第 n 个盒子放 $2n-1$ 和 $2n$ 这两个数。

如果我们在 n 个盒子里随意抽出 $n+1$ 个数。我们马上看到一定有一个盒子是被抽空的。而被抽空的盒子里的两个数一定是连续的，因此在我们任意选择的这 $n+1$ 个数中必有两个数是连续数，很明显的连续数是互质的。

推论2.7 得证 □

▲ 构造抽屉的方法

运用抽屉原理的核心是分析清楚问题中，哪个是物件，**哪个是抽屉**。例如，属相是有 12 个，那么任意 37 个人中，至少有一个属相是不少于 4 个人。这时将属相看成 12 个抽屉，则一个抽屉中有 $\frac{37}{12}$，即 3 余 1 ，余数不考虑，而向上考虑取整数，所以这里是 $3+1=4$ 个人，但这里需要注意的是，前面的余数 1 和这里加上的 1 是不一样的 。

因此，在问题中，**较多的一方就是物件**，较少的一方就是抽屉，比如上述问题中的属相 12 个，就是对应抽屉，37 个人就是对应物件，因为 37 相对 12 多。这样我们设抽屉个数为 m，物件个数为 n ，**也就意味着一定有** $\lceil \frac{m}{n} \rceil$ **个元素满足同样的性质**。我们通常利用这一点性质来证明一定有解。

▲ 最差原则

即考虑所有可能情况中，最不利于某件事情发生的情况。也就是最差的情况都能被满足，那么其余所有的情况都比最差的要好，连最差的都能满足，则说明其余所有情况也都能满足，也就是任意情况都能满足

Example A 求职

有 300 人到招聘会求职，其中软件设计有 100 人，市场营销有 80 人，财务管理有 70 人，人力资源管理有50人。那么至少有多少人找到工作才能保证一定有 70 人找的工作专业相同呢?

此时我们考虑的**最差情况**为：软件设计、市场营销和财务管理各录取 69 人，人力资源管理的 50 人全部录取，则此时再录取 1 人就能保证有 70 人找到的工作专业相同。因此至少需要 $69 \times 3 + 50 + 1 = 258$ 人。

根据 **定理2.3** 推导： $m \times n + 1$ 个人的时候必有 $m + 1$ 个人找到的工作专业相同，所以是要求出 $m \times n + 1$ 的人数，已知 $n = 3$，$m + 1 = 70$ 。考虑到人力资源专业只有 50 人，得出 $m \times n + 1 = (69 \times 3 + 50) + 1 = 258$ 人。

Example B 强迫症

一个抽屉里有 20 件衬衫，其中 4 件是蓝的， 7 件是灰的， 9 件是红的，则应从中**随意取出多少件**才能保证有 5 件是同颜色的?

根据抽屉原理，n 个抽屉， $k \times n + 1$ 件物件，则至少有一个抽屉中有 $k + 1$ 件物件。若根据抽屉原理的推论直接求解，此时 k=4，n=3，则应抽取 $3 \times 4 + 1 = 13$ 件才能保证有 5 件同色。其实不能这么考虑，因为原本的意思是差的那一个物件放到那个抽屉里都行，但是蓝色抽屉实际上是不能再放入的，所以本问题的模型实际上和鸽巢原理有些不同。在解决该问题时，应该考虑最差原则，连续抽取过程中抽取出 4 件蓝色的衬衣，取走后，问题变成有灰色和红色构成相同颜色的情况，这时，$n = 2$，$k + 1 = 5$，$k = 4$ 。故应取 $4 + 4 \times 2 + 1 = 13$ 件。

问题分析：该情况下鸽巢原理的推论不再适用，由于蓝色的衬衫只有 4 件，而题目中要求有 5 件是同色的，导致 4 件蓝色衬衫都被抽取出这一最差情况的存在，所以应该先考虑最差情况，然后在此基础上再运用鸽巢原理。

▲ 竞赛例题选讲

Problem A Halloween treats（POJ 3370）

给出一个含有 n 个数字的序列，要找一个连续的子序列，使他们的和一定是 c 的倍数，若不存在，则输出 `no sweetws`。$1 \leq c \leq n \leq 10^5$

Solution

首先预处理序列的前缀和 `sum[i]` 。

根据抽屉原理，以 sum 数组构造一个抽屉 $drawer$ 数组，其保存的是最先出现的 `sum[i] % c` 的下标，当 sum 的一个元素**第二次放入重复的抽屉时**，说明 **出现了一个和第一次放入抽屉的元素性质相同的元素** 。我们假设第一个 被放入该抽屉的元素的下标为 i ，第二次被放入重复的抽屉元素的下标是 j 。

此时：

- 若 `sum[i]` 是 c 的倍数，即 `sum[i] % c == 0` ，我们直接输出前 i 项就是答案，或者前 `j` 项也是答案，它们都有相同的性质：是 c 的倍数。
- 若 `sum[i]` 不是 c 的倍数，则此时， `sum[i] % c == sum[j] % c` 且 $i! = j$ ，则说明前 i 项的个与前 j 项的和模 c 同余，也就意味着 $sum[j] - sum[i] \equiv 0 \pmod{c}$，也就是 从第 $i + 1$ 项到第 j 项的和是 c 的倍数。

如何证明一定存在答案呢?

因为我们求的是 $\%c$ 相同的两个点，$\%c$ 的值一共只有 $0 \cdots c - 1$ ，c 种答案，但是我们一共有 $n \geq c$ 个数，根据抽屉原理，如果 $n = c$ ，则一定存在一个数 $x\%c = 0$ 是满足题意的答案，直接输出即可（这种情况下 n 个数构成模 c 的简化剩余系） ，或者存在两个数同余。 若 $n > c$ 则一定存在两个数同余。

同时得出：

推论2.7： 一个由 n 个数构成的数列，总能找到若干个连续的数，使它们之和能被 n 整除（就是上面 $n = c$ 的情况）

Code

```cpp
typedef long long ll;
typedef int itn;
const int N = 500007, M = 5000007, INF = 0x3f3f3f3f;
const ll LINF = 4e18 +7;
int n, m;
ll c, a[N], sum[N];
int drawer[N];
int main()
{
    while(scanf("%lld%d", &c, &n) != EOF && c | n) {
        for(int i = 1; i <= n; ++ i)
            drawer[i] = -1;
        drawer[0] = 0;
        sum[0] = 0;
        for(int i = 1; i <= n; ++ i) {
            scanf("%lld", &a[i]);
            sum[i] = sum[i - 1] + a[i];
        }
        for(int i = 1; i <= n; ++ i) {
            if(drawer[sum[i] %c] != -1) {
                for(int j = drawer[sum[i] % c] + 1; j < i; ++ j)
                    printf("%d ", j);
                printf("%d\n", i);
                break;
            }
            drawer[sum[i] % c] = i;
        }
    }
    return 0;
}
```

Problem B Flowers （ 2019-ICPC沈阳 - L ）

给你 n 种花和每种花的数量，每束花要 m 朵，每束花里不能有同种的花，问最多能准备几束花。

Solution

由于每束花里的花的种类一定不同，那么据抽屉原理，每种花能用的数量，一定小于等于花束的数量，因为一旦大于，就存在一个花束中至少有一种花的数量大于 1 。题目要求的是最多能有几个花束，很自然地想到二分，我们二分花束的数量，判断的时候，假设当前二分到一共可以有 mid 束花，则一共**需要** $mid \times m$ **束花**，我们根据上面的分析，每种花最多可以使用 $\min\{a[i], mid\}$ 枝，我们就可以计算当前最多能**提供** $\sum_{i=1}^{n} \min\{a[i], mid\}$ **枝花**，我们只需要判断二者的关系即可，也就是能满足，大于等于即为 `true` ，不够即为 `false` 。

Code

```cpp
typedef long long ll;
typedef int itn;
const int N = 500007, M = 5000007, INF = 0x3f3f3f3f, mod = 1e9 + 7;
const ll LINF = 4e18 +7;

int n, t;
ll m;
ll a[N];
```

```cpp
bool check(ll x)
{
    ll res = 0;
    for(int i = 1; i ≤ n; ++ i)
        res += min(a[i], x);
    return res ≥ x * m;
}
int main()
{
    scanf("%d", &t);
    while(t -- ) {
        scanf("%d%lld", &n, &m);
        ll sum = 0;
        for(int i = 1; i ≤ n; ++ i) {
            scanf("%lld", &a[i]);
            sum += a[i];
        }
        ll l = 0, r = sum / m, ans = 0;
        while(l ≤ r) {
            ll mid = l + r >> 1ll;
            if(check(mid)) l = mid + 1, ans = mid;
            else r = mid - 1;
        }
        printf("%lld\n", ans);
    }
    return 0;
}
```

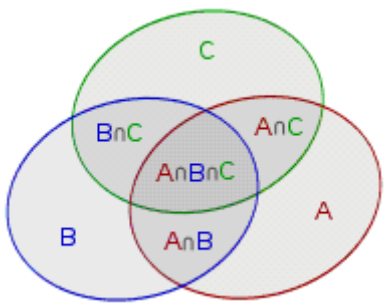
0x03 容斥原理

容斥原理是一种应用在集合上的较常用的计数方法，其基本思想是：先不考虑重叠的情况，把包含于某内容中的所有对象的数目先计算出来（**容**），然后再把计数时重复计算的数目排斥出去（**斥**），使得计算的结果既无遗漏又无重复。

容斥原理核心的计数规则可以归为一句话：**奇加偶减**。

假设被计数的有 A、B、C 三类，那么，A、B、C 类元素个数总和 $=$ A 类元素个数 $+$ B 类元素个数 $+$ C 类元素个数 $-$ 既是 A 又是 B 的元素个数 $-$ 既是 A 又是 C 的元素个数 $-$ 既是 B 又是 C 的元素个数 $+$ 既是 A 又是 B 且是 C 的元素个数。

即：$A \cup B \cup C = A + B + C - AB - BC - AC + ABC$



当被计数的种类被推到 n 类时，其统计规则遵循**奇加偶减**。

- **竞赛例题选讲**

Problem A Co-prime（HDU 4135）

求 $[a, b]$ 区间与 n 互质的数的个数，其中 $1 \leq a, b, n \leq 2^{31}$。

Solution

容斥定理最常用于**求解 $[a,b]$ 区间与 n 互质的数的个数问题**，该问题可以视为求 $[1,b]$ 区间与 n 互质的个数减去 $[1,a-1]$ 区间内与 n 互质的个数。

那么我们这里只需要考虑它的一个子问题：

求小于等于 m 且 n 互素的数的个数。

我们知道当 m 等于 n ，就是一个简单的欧拉函数问题，但是一般 m 都不等于 n ，我们考虑将 n 质因数分解。

我们首先分析最简单的情况：当 n 为素数的幂，即 $n=p^k$ 时，那么显然答案就等于 $m-\frac{m}{p}$ （其中 $\frac{m}{p}$ 表示的是 p 的倍数，$1\sim m$ 中去掉 p 的倍数，剩下的就都是与 n 互素的数了）

然后再来讨论 n 是两个素数的幂的乘积，即 $n=p_1^{k_1}\times p_2^{k_2}$，那么我们需要做的就是找到 p_1 的倍数和 p_2 的倍数，并且要减去 p_1 和 p_2 的公倍数，我们发现这实际上就是容斥原理，所以这种情况下答案为：$m-(\frac{m}{p_1}+\frac{m}{p_2}-\frac{m}{p_1\times p_2})$。

这里的 $+$ 就是 **容**， $-$ 就是 **斥**，并且 **容** 和 **斥** 总是交替进行的（一个的加上，两个的减去，三个的加上，四个的减去），而且可以推广到 n 个元素的情况,如果 n 分解成 s 个素因子，也同样可以用容斥原理求解。

容斥原理其实是枚举子集的过程，常见的枚举方法为 dfs ，也可以采用二进制法（0 表示取，1 表示不取）。

例如我们求 $[1,9]$ 中和 6 互素的数的个数，这时6 分解的素因子为 2 和 3 。

$ans=9-(\frac{9}{2}+\frac{9}{3})+\frac{9}{6}=3$，其中，$ans$ 分为三部分， 0 个数的组合， 1 个数的组合，2 个数的组合。

答案 = （$1\sim b$ 的元素个数） $-$ （$1\sim a-1$ 的元素个数） $-$ （$1\sim b$ 中与 n 不互质的数的个数） $+$ （$1\sim a-1$ 中与 n 互质的数的个数）

Code

```cpp
const int N = 10005;
ll n, primes[N], cnt, factor[N], num;
bool vis[N];
inline void get_primes(int n)
{
    for(register int i = 2;i ≤ n;i ++) {
        if(!vis[i]) primes[ ++ cnt] = i;
        for(register int j = 1;j ≤ cnt && i * primes[j] ≤ n; ++ j) {
            vis[i * primes[j]] = 1;
            if(i % primes[j] == 0) break;
        }
    }
}
void get_factor(int n) {
    num = 0;
    for (ll i = 1; primes[i] * primes[i] ≤ n && i ≤ cnt; i ++ ) {
        if (n % primes[i] == 0) { //记录n的因子
            factor[num ++ ] = primes[i];
            while (n % primes[i] == 0)
                n /= primes[i];
        }
    }
    if (n ≠ 1) //1既不是素数也不是合数
        factor[num ++ ] = n;
}
ll solve(ll m, ll num) {
    ll res = 0;
    for (ll i = 1; i < (1 << num); i++) {
        ll sum = 0;
        ll temp = 1;
        for (ll j = 0; j < num; j++) {
```

```
            if (i & (1 << j)) {
                sum ++ ;
                temp *= factor[j];
            }
        }
        if (sum % 2) res += m / temp;
        else res -= m / temp;
    }
    return res;
}
int kcase, t;
int main() {
    get_primes(N - 1);
    scanf("%d", &t);
    while(t -- ) {
        ll a, b, n;
        scanf("%lld%lld%lld", &a, &b, &n);
        get_factor(n);
        //容斥定理，奇加偶减，
        ll res = (b - (a - 1) - solve(b, num)) + solve(a - 1, num);
        printf("Case #%d: %lld\n", ++ kcase, res);
    }
    return 0;
}
```

Problem B Helping Cicada（LightOJ - 1117）

求 $[1,n]$ 中不能被 m 个数整除的个数。

Solution

我们知道对于任意一个数 k，在 $[1,n]$ 中有 $\lfloor \frac{n}{k} \rfloor$ 个数是 k 的倍数，也就是能被 k 整除，故 $ans = n - \sum_{i=1}^{n} \lfloor \frac{n}{a[i]} \rfloor$ 。

我们再来考虑两个数 a,b 的情况，因为 a,b 的公倍数 $lcm(a,b)$，即被 a 整除，又被 b 整除，所以减了两次。所以最后的答案要加上 $\lfloor \frac{n}{lcm(a,b)} \rfloor$。对于三个数 a,b,c 来说，答案就要减去 $\lfloor \frac{n}{lcm(a,b,c)} \rfloor$，以此类推

最后拓展到 m 个数的情况，我们发现需要使用容斥定理。

根据容斥定理的**奇加偶减**，对于 m 个数来说，其中的任意 2、4、$\cdots$、$2k$ 个数就要**减去**他们最小公倍数能组成的数，1、3、$\cdots$、$2k+1$ 个数就要**加上**他们的最小公倍数，对于每一个数来说，我们都有选或不选两种情况，因此 m 个数就有 2^m 种情况，也就是从0（啥也不选）到 2^m-1，对应二进制就是 m 个 1，也就意味着 我们选择了 m 个数。这种方法叫做状态压缩，我们接用状态压缩来枚举所有的可能状态，依次使用位运算判断当前的状态选择了多少个数，然后再进行奇加偶减即可。

$sum =$ （从 m 中选 1 个数得到的倍数的个数） $-$ （从 m 中选 2 个数得到的倍数的个数） $+$ （从 m 中选 3 个数得到的倍数的个数） $-$ (从 m 中选 4 个数得到的倍数的个数）.....

那么能被整除的数的个数就是 sum，不能被整除的数的个数就是 $n-sum$ 。

Code

```
const int N = 10005;
ll LCM(ll a, ll b) {
    return a / __gcd(a, b) *b;
}
int m, kcase;
ll n, a[N];
int main()
{
```

```
    int t;
    scanf("%d", &t);
    while(t -- ) {
        scanf("%lld%d", &n, &m);
        for(int i = 0; i < m; ++ i)
            scanf("%lld", &a[i]);
        ll sum = 0;
        for(int i = 0; i < (1 << m); ++ i) {
            ll lcm = 1;
            ll cnt = 0;
            for(int j = 0; j < m; ++ j) {
                if(i >> j & 1) {//如当前的状态i选择了第j个数
                    lcm = LCM(lcm, a[j]);//那就选第j个数
                    cnt ++ ;
                }
            }
            if(cnt != 0) {
                if(cnt & 1) sum += n / lcm;//奇加
                else sum -= n / lcm;//偶减
            }
        }
        printf("Case %d: %lld\n", ++ kcase, n - sum);
    }
    return 0;
}
```

Problem C 硬币购物 （[Luogu P1450 [HAOI2008]](https://www.luogu.com.cn/problem/P1450)）

共有 4 种硬币。面值分别为 c_1, c_2, c_3, c_4 。

某人去商店买东西，去了 n 次，对于每次购买，他带了 d_i 枚 i 种硬币，想购买 s 的价值的东西。请问每次有多少种付款方法。

Solution

看起来像是一个多重背包求方案数的模板题，但是由于有 n 组，所以每次都求一次多重背包绝对会超时。

显然多重背包即有限制的方案数，直接计算有限制的方案数比较难，考虑**正难则反**：

有限制的方案数 = 无限的方案数 - 超过限制方案数

首先无限制的方案数显然就是完全背包，我们可以先用完全背包预处理出所有的方案数，即 `f[i]` 表示购买价值为 `i` 的方案数。

考虑减去超过显示的即不合法的部分，考虑使用容斥原理。

首先考虑一种硬币超额使用的方案数怎么计算。若第 j 种硬币超额使用，即超过了原定的 d_j 个硬币的限制，我们可以先强制选了d_j+1个第j 种硬币，这样剩下的价值里， 4 种硬币随便选，这样就能保证第 j 种硬币一定超额使用，第 j 种硬币超额的不合法的方案数就等于 $f[s-(d_j+1)\times c_j]$。

然后我们只需要使用容斥原理奇加偶减即可。加上一种硬币不合法的方案数，减去两种硬币不合法的方案数，加上三种硬币不合法的方案数 $\cdots$ 二进制枚举子集即可。

Code

```
// Problem: P1450 [HAOI2008]硬币购物
// Contest: Luogu
// URL: https://www.luogu.com.cn/problem/P1450
```

```cpp
// Memory Limit: 125 MB
// Time Limit: 1000 ms
//
// Powered by CP Editor (https://cpeditor.org)

#include <bits/stdc++.h>
#define int long long
using namespace std;
const int N = 1e5 + 7;
int n, m, s, t, k, ans;
int f[N];
int c[50], d[50];

void pre_work(int n)
{
    f[0] = 1;
    for (int i = 1; i ≤ 4; ++ i)
        for (int j = c[i]; j < n; ++ j)
            f[j] += f[j - c[i]];
}

void solve()
{
    scanf("%lld%lld%lld%lld%lld", &d[1], &d[2], &d[3], &d[4], &s);

    int ans = f[s], now;
    for (int i = 1; i ≤ (1 << 4) - 1 ; ++ i) {
        now = s;

        int tmp, j, k;
        for (tmp = i, j = 1, k = 0; tmp; tmp >>= 1, ++ j) {
            if(tmp & 1) {
                k ^= 1;
                now -= (d[j] + 1) * c[j];
            }
        }
        if(now ≥ 0)
            k ? ans -= f[now] : ans += f[now];
    }
    printf("%lld\n", ans);
}

signed main()
{
    scanf("%lld%lld%lld%lld%lld", &c[1], &c[2], &c[3], &c[4], &n);
    pre_work(N - 7);
    while (n -- ){
        solve();
    }
    return 0;
}
```

Hint

我们一般根据研究对象的有无，将组合数学分为**排列问题**和**组合问题**。

其根本不同是排列问题与元素顺序**有关**，组合问题与元素顺序**无关**。

在排列与组合问题中，经常会出现计数问题，解决计数问题的思路一般有以下三种：

1）只取需要的。将各种符合条件的情形枚举出来，再利用**加法原理**求和。

2）先取后排。将各步符合条件的排列或组合计算出来，再根据**乘法原理**求积。

3）先全部取，再减去不要的。利用**容斥定理**，将各种符合条件的情形枚举出来，再减去不符合条件的。

0x10 排列

0x11 排列

定义： 从 n 个元素的集合 S 中，**有序的**选出 r 个元素，叫做 S 的一个 r 排列，不同的排列总数记作：A_n^r 或 $P(n,r)$

如果两个排列所含元素不全相同，或所含元素相同但顺序不同，就会被认为是不同的排列。

可重排列

从 n 个不同元素可重复的取出 m 个元素，按照一定顺序排成一列，叫做相异元素可重复排列。

相异元素可重复排列的方案数为：n^m （取 m 次，每次都有 n 种取法）

例如：从 1、2、3、4、5 中任取三个出来组成一个三位数，有 $P(5,3)=60$ 种情况，如果每个数字可以重复使用，则有：$5^3=125$ 种情况。

不可重排列

不可重排列是指在 n 个不同元素中选 r 个元素**按照顺序**排成一列。

选排列： 从 n 个不同元素取出 r 个元素，按照一定顺序排成一列，当 $r<n$ 时，叫做从 n 个不同元素取出 r 个不同元素的一种选排列。

我们可以一步一步的考虑这个问题，首先从 n 个小朋友中抽取一个放在第一个位置有 n 种方法。之后从剩下的 $n-1$ 个小朋友中挑选一个放在第二个位置即有 $n-1$ 种方法。以此类推，根据乘法原理我们可以得到

$$\mathrm{A}_n^r=n\times(n-1)\times(n-2)\times\cdots\times(n-r+1)=\frac{n!}{(n-r)!}$$

注：$0!=1$。

排列数的性质

性质11.1： $\mathrm{A}_n^m=n\mathrm{A}_{n-1}^{m-1}$ （可以理解为 某特定位置 先安排，再安排其余位置。）

性质11.2： $\mathrm{A}_n^m=m\mathrm{A}_{n-1}^{m-1}+\mathrm{A}_{n-1}^m$（可理解为：含特定元素的排列有$m\mathrm{A}_{n-1}^{m-1}$，不含特定元素的排列为 A_{n-1}^m）

Problem A 输出排列

输入两个整数 n、r，输出 $P(n,r)$ 的所有方案

Solution

n个数里选 r 个数，直接爆搜即可。

```
1 int n, r, a[N], vis[N];
```

```
void dfs(int i) {//n个数里选 r 个数
    if (i == r) {
        for (int j = 1; j ≤ r; ++ j)
            printf("%d", a[j]);
        printf("%d\n", a[r]);
        return;
    }
    for (int j = 1; j ≤ n; ++ j) {
        if (vis[j] == 0) {
            vis[j] = true;
            a[i] = j;
            dfs(i + 1);
            vis[j] = false;
        }
    }
}
int main() {
    memset(vis, 0, sizeof vis);
    scanf("%d%d", &n, &r);
    dfs(0);
    return 0;
}
```

0x12 全排列

定义： 从 n 个不同元素取出 r 个元素，按照一定顺序排成一列，当 $r=n$ 时，叫做 n 个不同元素的**全排列**。

全排列的方案数：$\mathrm{A}_n^n=n!$

C++ 中，头文件`<algorithm>`里的`next_permutation()`函数，可产生字典序的全排列。

例如：给出从一组全排列的情况，使用`next_permutation()`函数可以生成下一种的全排列的情况，因此一般先使用`sort()`进行排序，即可生成所有全排列的情况。

```
int a[N];
int main(){
    int n;
    scanf("%d",&n);
    for(int i=0;i<n;i++)
        scanf("%d",&a[i]);


    sort(a,a+n);
    do{
        for(int i=0;i<n;i++)
            printf("%d ",a[i]);
        printf("\n");
    }while(next_permutation(a,a+n));
    return 0;

}
```

不全相异排列

（1）不全相异元素的选排列

若在 n 个元素中，有 n_1 个元素彼此相同，n_2 个元素彼此相同，...，n_m 个元素彼此相同，且 $n_1+n_2+...+n_m=r$，则这 n 个元素选出 r 个的选排列叫做**不全相异元素的选排列**。

排列数计算公式为：$\frac{A(n,r)}{(n_1!\times n_2!\times\cdots\times n_m!)}$

（2）不全相异元素的全排列

若在 n 个元素中，有 n_1 个元素彼此相同，n_2 个元素彼此相同，...，n_m 个元素彼此相同，且 $n_1+n_2+\ldots+n_m=n$，则这 n 个元素的全排列叫做不全相异元素的全排列。

排列数计算公式为：$\frac{n!}{(n_1!\times n_2!\times\cdots\times n_m!)}$

0x13 错位排列与圆排列

定义： 设 $(a_1,a_2,\ldots,a_n)$ 是 $\{1,2,\ldots,n\}$ 的一个全排列，若对任意的 $i\in\{1,2,\ldots,n\}$ 都有 $a_i\neq i$，则称 $(a_1,a_2,\ldots,a_n)$ 是 $\{1,2,\ldots,n\}$ 的错位排列。

用 D_n 表示 $\{1,2,\ldots,n\}$ 的错位排列的个数，有：$D_n=n!\times(1-\frac{1}{1}!+\frac{1}{2}!-\frac{1}{3}!+\cdots+\frac{(-1)^n}{n}!)$

错排公式

$f[0]=1,f[1]=0,f[2]=1$

$f[n]=(n-1)*(f[n-1]+f[n-2])$

圆排列

从 n 个不同元素中选取 r 个元素，不分首尾地围成一个圆圈的排列叫做圆排列，其排列方案数为：$\frac{P(n,r)}{r}$

当 $r=n$ 时，则为圆排列的全排列，其排列方案数为：$\frac{n!}{n}=(n-1)!$

圆排列实际上就是群论里的**旋转置换**，也就是通过旋转任意度数，若相同，则属于同一种方案。我们可以将其想象成一个轮子，怎么转都是一样的，**如图11.1**，`1 2 3 4 5` 和 `5 4 1 2 3` 都算是一种方案，

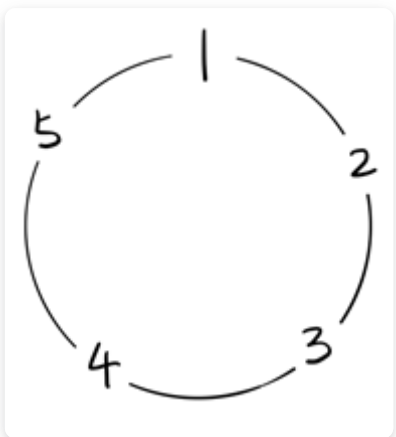


图11.1

例题：有男女各 5 人，其中有 3 对夫妇，沿 10 个位置的圆桌就座，若每对夫妇都要坐在相邻的位置上，有多少种坐法？

分析：先让 3 对夫妇中的妻子和其他 4 人就坐，根据圆排列公式，共有 $(n-1)!=6!$ 种坐法，然后每位丈夫都可以做到自己的妻子左右两边，因此共有 $6!\times2\times2\times2=5760$ 种坐法。

0x20 组合

0x21 组合

定义： 从 n 个元素的集合 S 中，**无序**的选出 r 个元素，叫做 S 的一个 r 组合。

如果两个组合中，至少有一个元素不同，它们就被认为是不同的组合（不考虑顺序，元素相同顺序不同仍然是同一个组合）。

不可重组合数

所有不同组合的个数，叫做组合数，记作：C_n^r 或 $\binom{n}{r}$

由于每一种组合都可以扩展到 $r!$ 种排列，而总排列为 $P(n,r)$

所以组合数：$C_n^r=\binom{n}{r}=\frac{P(n,r)}{r!}=\frac{n(n-1)(n-2)\cdots(n-r+1)}{r!}=\frac{n!}{(n-r)!r!},r\leqslant n$

特别的，规定：$C(n,0)=1$

可重复组合数

从 n 个不同的元素中，无序的选出 r 个元素组成一个组合，且允许这 r 个元素可以**重复使用**，则称这样的组合为可重复组合。

其组合数记为：$H_n^r=C_{n+r-1}^r=\frac{n+r-1}{r!(n-1)!}$

不相邻组合数

从 $A=\{1,2,\ldots,n\}$ 中选取 m 个不相邻的组合，其组合数为：C_{n-m+1}^m

Example A

① 一班有 10 名同学，二班有 8 名同学，现每个班级要选出 2 名学生参加一个座谈会，求有多少种选法?

根据组合数与乘法原理，共有：$C(10,2)\times C(8,2)=1260$ 种

② 某班有 10 名同学，有 4 名女同学，现要选出 3 名学生，其中至少有一名女同学，求有多少种选法?

根据组合数与加法原理，共有：
$C(4,1)\times C(6,2)+C(4,2)\times C(6,1)+C(4,3)\times C(6,0)=60+36+4=100$ 种

0x22 组合数的性质

性质22.1： $C_n^m=C_n^{n-m}$

性质22.2： $C_n^m=C_{n-1}^m+C_{n-1}^{m-1}$

性质22.3： $C_n^{m+1}=\frac{n-m}{m+1}\times C_n^m$

性质22.4： $C_n^0+C_n^1+\cdots+C_n^n=2^n$

性质22.5： $C_n^0+C_n^2+C_n^4+\cdots=C_n^1+C_n^3+C_n^5\cdots=2^{n-1}$

性质22.6：

$$\begin{aligned}&(C_1^{2n})+(C_3^{2n})+\cdots+(C_{2n-1}^{2n})\\=&(C_0^{2n-1})+(C_1^{2n-1})+(C_2^{2n-1})+(C_3^{2n-1})+\cdots+(C_{2n-2}^{2n-1})+(C_{2n-1}^{2n-1})\\=&2^{2n-1}\end{aligned}$$

性质22.7： C_n^m 为奇数时有 $n\ \&\ m=n$

性质22.8： C_n^m 的值一定是自然数，因为连续 m 个自然数的积一定被 $m!$ 整除

性质22.9： $C_n^k\times C_k^r=C_n^r\times C_{n-r}^{k-r},\ (k\geq r)$

性质22.10： $C_{n+r+1}^r=C_{n+r-1}^{r-1}+C_{n+r-2}^{r-2}+\cdots+C_{n+1}^1+C_n^0$

0x23 求组合数

0x23.1 加法递推 $O(n^2)$

Problem A 求组合数 I （AcWing 885）

给定 n 组询问，每组询问给定两个整数 a,b ，请你输出 $C_a^b \mod (10^9+7)$ 的值。

$1 \le n \le 100000, 1 \le b \le a \le 2000$

Solution

利用 **性质12.2.2**，边界条件为 $C_n^0 = C_n^n = 1$ 。（其实就是**杨辉三角 / 帕斯卡三角**）

Code

```cpp
const int N = 2500, mod = 1e9 + 7;
const double PI = acos(-1.0);

int n, m;
int a[N];
int c[N][N];
//C(n, m) = C(n - 1, m) + C(n - 1, m - 1);

void init()
{
    c[1][0] = c[1][1] = 1;
    for(int i = 2; i < N; ++ i) {
        c[i][0] = 1;
        for(int j = 1; j < N; ++ j)
            c[i][j] = (1ll * c[i - 1][j] + c[i - 1][j - 1]) % mod;
    }
}

int main()
{
    //freopen("popcorn.in","r",stdin);
    init();
    scanf("%d", &n);
    while(n -- ){
        int a, b;
        scanf("%d%d", &a, &b);
        printf("%d\n", c[a][b]);
    }
    return 0;
}
```

0x23.2 乘法递推 $O(n)$

$C_n^m = \frac{n-m+1}{n} \times C_n^{m-1}$，边界条件$C_n^0 = 1$。必须先乘后除，不然可能除不开

可以利用性质1，减少部分运算

```cpp
c[0] = 1;
for( register int i = 1 ; i * 2 ≤ n ; i ++ )
    c[i] = c[n-i] = ( n - i + 1 ) * c[ i - 1 ] / i;
```

Problem B 求组合数 II （AcWing 886）

给定 n 组询问，每组询问给定两个整数 a,b ，请你输出 $C_a^b \mod (10^9+7)$ 的值。

$1 \le n \le 10000, 1 \le b \le a \le 10^5$

Solution

直接用公式 $C_m^n = \frac{m!}{(m-n)!\times n!}$ 求，把除转化为逆元。

Code

```cpp
const int N = 1e6 + 7, mod = 1e9 + 7;
int n, m, k, t;
int inv[N];
int fact[N], infact[N];

void init (int n)
{
    fact[0] = infact[0] = inv[0] = inv[1] = 1;
    for (int i = 2; i ≤ n; ++ i)
        inv[i] = 1ll * (mod - mod / i) * inv[mod % i] % mod;
    for (int i = 1; i ≤ n; ++ i) {
        fact[i] = 1ll * fact[i - 1] * i % mod;
        infact[i] = 1ll * infact[i - 1] * inv[i] % mod;
    }
}

int C(int n, int m)
{
    if(n < m) return 0;
    if(m == 0 || n == m) return 1;
    return 1ll * fact[n] * infact[m] % mod * infact[n - m] % mod;
}

int main()
{
    init(N - 2);
    scanf("%d", &n);
    for (int i = 1; i ≤ n; ++ i) {
        int a, b;
        scanf("%d%d", &a, &b);
        cout << C(a, b) << endl;
    }
    return 0;
}
```

0x23.3 Lucas定理

Lucas定理： 若p是质数，则对于任意的整数 $1 \leq m \leq n$ ，有：

$\binom{n}{m} \equiv \binom{n \bmod p}{m \bmod p} \cdot \binom{\lfloor \frac{n}{p} \rfloor}{\lfloor \frac{m}{p} \rfloor} \pmod p$

Lucas定理推论

当将 n 写成 p 进制：$n = a[n]a[n-1]\ldots a[0]$，将 m 写成 p 进制：$m = b[n]b[n-1]\ldots b[0]$ 时，有：

$\binom{a[n]}{b[n]} \times \binom{a[n-1]}{b[n-1]} \times \ldots \times \binom{a[0]}{b[0]} \equiv \binom{n}{m} \ mod \ p)$

Problem C 求组合数 Ⅲ （AcWing 887）

给定 n 组询问，每组询问给定三个整数 a, b, p ，其中 p 是质数，请你输出 $C_a^b \mod p$ 的值。

$1 \leq n \leq 20, 1 \leq b \leq a \leq 10^{18}, 1 \leq p \leq 10^5$

Solution

卢卡斯定理模板题

```cpp
const int N = 10007, M = 500007, Mod = 1000003, INF = 0x3f3f3f3f;
ll n, m, p;

ll qpow(ll a, ll b, ll mod)
{
    ll res = 1;
    while(b) {
        if(b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

ll C(ll a, ll b, ll p)
{
    if(a < b)return 0;

    ll down = 1, up = 1;
    for(int i = a, j = 1; j ≤ b; i -- , ++ j) {
        up = up * i % p;
        down = down * j % p;
    }
    return up * qpow(down, p - 2, p) % p;
}

ll lucas(ll a, ll b, ll p)
{
    if(a < p && b < p) return C(a, b, p);
    return C(a % p, b % p, p) * lucas(a / p, b / p, p) % p;
}

int t;

int main()
{
    scanf("%d", &t);
    while (t -- ) {
        //C(down, up) % p;
        //C(大, 小) % p;
        scanf("%lld%lld%lld", &n, &m, &p);
        printf("%lld\n", lucas(n, m, p) % p);
    }
    return 0;
}
```

0x23.4 扩展卢卡斯

卢卡斯定理适用于 p 是素数的情况，但当 p 不是素数时，可以将其分解质因数，将组合数按照卢卡斯定理的方法求 p 的质因数的模，然后用中国剩余定理合并即可。

当需要计算 $C_m^n \ mod \ p, p = p_1^{q1} * p_2^{q2} * \ldots * p_k^{qk}$ 时，可以求出：$C_m^n \equiv ai(mod \ p_i^{qi})(1 < i < k)$

然后对于方程组：$x \equiv ai(mod \ p_i^{qi})(1 < i < k)$，可以求出满足条件的最小的 x，记为：x_0

那么有：$C_m^n \equiv x_0(mod \ p)$

但是，p_i^{qi} 并不是一个素数，而是某个素数的某次方，那么就需要计算 $C_m^n \ mod \ p^t, t \geqslant 2.$

对于 $C_m^n \ mod \ p^t, t \geqslant 2$，已知 $C_n^m = \frac{n!}{m!(n-m)!}$，因此若能计算出 $n! \ mod \ p^t$，就能计算出 $m! \ mod \ p^t$ 和 $(n-m!) \ mod \ p^t$

设 $\begin{cases} x = n! \ mod \ p^t \\ y = m! \ mod \ p^t \\ z = (n-m)! \ mod \ p^t \end{cases}$，那么答案就是 $x * inv(y, p^t) \times inv(z, p^t)$，其中 $inv(a, b)$ 代表计算 a 对 b 的乘法逆元

于是，问题就转换为如何计算 $n! \ mod \ p^t$

例如：$p = 3$，$t = 2$，$n = 19$，有：

$$
\begin{aligned}
n! &= 1 \times 2 \times 3 \times 4 \times 5 \times 6 \times 7 \times 8 \times \ldots \times 19 \\
&= (1 \times 2 \times 4 \times 5 \times 7 \times 8 \times \ldots \times 16 \times 17 \times 19) \times (3 \times 6 \times 9 \times 12 \times 15 \times 18) \\
&= (1 \times 2 \times 4 \times 5 \times 7 \times 8 \times \ldots \times 16 \times 17 \times 19) \times 36 \times (1 \times 2 \times 3 \times 4 \times 5 \times 6)
\end{aligned}
$$

后半部分是 $(\frac{n}{p})!$，递归即可。前半部分是以 p^t 为周期的

$(1 * 2 * 4 * 5 * 7 * 8) \equiv (10 * 11 * 13 * 14 * 16 * 17)(mod \ 9)$

下面是孤立的 19，可以知道孤立出来的长度不超过 p^t，直接计算即可。

对于最后剩下的 36，只要计算出 $n!$、$m!$、$(n-m)!$ 里含有多少个 p，设他们分别有 x、y、z 个 p，那么 $x-y-z$ 就是 C_m^n 中 p 的个数，直接计算即可。

Problem D【模板】扩展卢卡斯（luogu P4720 ）

求 $C_m^n \ mod \ p$，其中p不一定是质数

Solution

扩展卢卡斯模板

```cpp
#define gc getchar
void exgcd(ll a, ll b, ll &x, ll &y) {
    if (!b)
        return (void)(x = 1, y = 0);
    exgcd(b, a % b, x, y);
    ll tmp = x;
    x = y;
    y = tmp - a / b * y;
}
ll gcd(ll a, ll b) {
    if (b == 0)
        return a;
    return gcd(b, a % b);
}
inline ll INV(ll a, ll p) {
    ll x, y;
    exgcd(a, p, x, y);
    return (x + p) % p;
```

```
}
inline ll lcm(ll a, ll b) {
    return a / gcd(a, b) * b;
}
inline ll mabs(ll x) {
    return (x > 0 ? x : -x);
}
inline ll fast_mul(ll a, ll b, ll p) {
    ll t = 0;
    a %= p;
    b %= p;
    while (b) {
        if (b & 1LL)
            t = (t + a) % p;
        b >>= 1LL;
        a = (a + a) % p;
    }
    return t;
}
inline ll fast_pow(ll a, ll b, ll p) {
    ll t = 1;
    a %= p;
    while (b) {
        if (b & 1LL)
            t = (t * a) % p;
        b >>= 1LL;
        a = (a * a) % p;
    }
    return t;
}
inline ll read() {
    ll x = 0, f = 1;
    char ch = gc();
    while (!isdigit(ch)) {
        if (ch == '-')
            f = -1;
        ch = gc();
    }
    while (isdigit(ch))
        x = x * 10 + ch - '0', ch = gc();
    return x * f;
}
inline ll F(ll n, ll P, ll PK) {
    if (n == 0)
        return 1;
    ll rou = 1; //循环节
    ll rem = 1; //余项
    for (ll i = 1; i ≤ PK; i++) {
        if (i % P)
            rou = rou * i % PK;
    }
    rou = fast_pow(rou, n / PK, PK);
    for (ll i = PK * (n / PK); i ≤ n; i++) {
        if (i % P)
            rem = rem * (i % PK) % PK;
```

```
    }
    return F(n / P, P, PK) * rou % PK * rem % PK;
}
inline ll G(ll n, ll P) {
    if (n < P)
        return 0;
    return G(n / P, P) + (n / P);
}
inline ll C_PK(ll n, ll m, ll P, ll PK) {
    ll fz = F(n, P, PK), fm1 = INV(F(m, P, PK), PK), fm2 = INV(F(n - m, P, PK), PK);
    ll mi = fast_pow(P, G(n, P) - G(m, P) - G(n - m, P), PK);
    return fz * fm1 % PK * fm2 % PK * mi % PK;
}
ll A[1001], B[1001];
//x=B(mod A)
inline ll exLucas(ll n, ll m, ll P) {
    ll ljc = P, tot = 0;
    for (ll tmp = 2; tmp * tmp ≤ P; tmp++) {
        if (!(ljc % tmp)) {
            ll PK = 1;
            while (!(ljc % tmp)) {
                PK *= tmp;
                ljc /= tmp;
            }
            A[++tot] = PK;
            B[tot] = C_PK(n, m, tmp, PK);
        }
    }
    if (ljc ≠ 1) {
        A[++tot] = ljc;
        B[tot] = C_PK(n, m, ljc, ljc);
    }
    ll ans = 0;
    for (ll i = 1; i ≤ tot; i++) {
        ll M = P / A[i], T = INV(M, A[i]);
        ans = (ans + B[i] * M % P * T % P) % P;
    }
    return ans;
}
signed main() {
    ll n = read(), m = read(), P = read();
    printf("%lld\n", exLucas(n, m, P));
    return 0;
}
```

0x23.5 求大组合数（高精）

Problem E求组合数 IV （AcWing 888）

输入 a, b 求 C_a^b。

Solution

由于这里没有取模，所以答案就会非常的大，需要用到高精算法。并且我们在求阶乘的时候可以将阶乘分解质因数再乘。

```
const int N = 5010;
int primes[N], cnt;
```

```
int sum[N];
bool st[N];
//线性筛素数;
void get_primes(int n)
{
    for (int i = 2; i ≤ n; i ++) {
        if (!st[i]) primes[cnt ++] = i;
        for(int j = 0; primes[j] ≤ n / i; j ++) {
            st[primes[j] * i] = true;
            if (i % primes[j] == 0) break;
        }
    }
}
//得到n! 中p的次数;
int get(int n, int p)
{
    int res = 0;
    while(n) {
        res += n / p;
        n /= p;//n每次除以p;
    }
    return res;
}
//高精度乘法;
vector<int> mul(vector<int> a, int b)
{
    vector<int> c;
    int t = 0;
    for(int i = 0; i < a.size(); i ++) {
        t += a[i] * b;
        c.push_back(t % 10);
        t /= 10;
    }
    while(t) {
        c.push_back(t % 10);
        t /= 10;
    }
    return c;
}
int main() {
    int a, b;
    cin >> a >> b;

    get_primes(a);

    for (int i = 0; i < cnt; i ++) {
        int p = primes[i];
        sum[i] = get(a, p) - get(b, p) - get(a - b, p);//得到C[a][b]中p的次数，因为是幂的形式，所以是减;
    }

    vector<int> res;
    res.push_back(1);//要先放进去一个1;
    for(int i = 0; i < cnt; i ++)
        for(int j = 0; j < sum[i]; j ++)
```

```
            res = mul(res, primes[i]);
    for(int i = res.size() - 1; i ≥ 0; i --) printf("%d", res[i]);
    return 0;

}
```

0x23.6 约分求重数

约分之后，分母即会变为 1，借此将除法化为乘法，约分方法是计算 1 到 n 之间的任意一个质数在 $C(n,r)$ 的重数。

具体做法是对分子分母上的每个数分解质因子，用一个数组 `C[N]` 来记录重数，若分子上的数分解一个质因子 p，则 `C[p] ++`，反之若分母上的数分解出质因子 p，则 `C[p] --`，最后将每个质因子按其重数连乘即可。

将公式化为 $C_n^r=\frac{A(n,r)}{r!}=\frac{n!}{r!(n-r)!}$，通过直接计算质数 p 在 $n!$ 中的重数而得到数组 `C[N]`，质数 p 在自然数 n 中的重数是指自然数 n 的质因数分解式质数 p 出现的次数，质数 p 在 $n!$ 的重数为：$n\div p+n\div p^2+n\div p^3+...$，根据公式：$n\div p^{k+1}=n\div p^k\div p$，可以递推的求出 p 在 $n!$中的重数。

例如：$72=2*2*2*3*3$，质数 2 在 72 的重数是 3，质数 3 在 72 的重数是 2。$n=1000$，$p=3$ 时，有 $1000\div 3+1000\div 9+1000\div 27+1000\div 81+1000\div 243+1000\div 729=333+111+37+12+4+1=498$，因此 $1000!$能被 3^{498} 整除，但不能被 3^{499} 整除，使用递推公式后，有：$333\div 3=111,111\div 3=37,37\div 3=12,12\div 3=4,4\div 3=1$

程序实现时，先求出 1 到 n 间所有质数，再对每个质数求重数，从而计算从 $n-r+1$ 到 n 的因子的重数与从 1 到 r 的因子的重数，前者减去后者，$C[i]$ 中所存储的即为约分后质数因子的重数，再利用高精度加法，将答案存储，最后倒序输出即可。

Code

```
const int N = 30000;
vector<int> prime, C;
bool vis[N];
int res[10];
void Get_Prime() {
    memset(vis, true, sizeof(vis));
    for (int i = 2; i ≤ N; i++) {
        if (vis[i]) {
            prime.push_back(i);//存储质数
            C.push_back(0);//当前质数的重数为0
            for (int j = i * i; j ≤ N; j += i) //筛除所有以i为因子的数
                vis[j] = false;
        }
    }
}
void Add(int n, int p) { //记录重数个数
    for (int i = 0; i < prime.size() && prime[i] ≤ n; i++) {
        while (!(n % prime[i])) {
            n /= prime[i];
            C[i] += p;
        }
    }
}
int main() {
    Get_Prime();//打表获取质数
    int n, r;
    scanf("%d%d", &n, &r);
    if (r > n - r) //根据公式C(n,r)=C(n,n-r)简化计算
        r = n - r;
```

```
    for (int i = 0; i < r; i++) {
        Add(n - i, 1); //将n-r+1到n的因子加到C中去
        Add(i + 1, -1); //将1到r的因子从C中减去
    }
    memset(res, 0, sizeof(res));
    res[0] = 1;
    for (int i = 0; i < prime.size(); i++) { //枚举所有质数
        for (int j = 0; j < C[i]; j++) { //枚举对应质数的重数
            for (int k = 0; k < 10; k++)
                res[k] *= prime[i];
            for (int k = 0; k < 10; k++) { //高精存储答案
                if (k < 9)
                    res[k + 1] += res[k] / 10;
                res[k] %= 10;
            }
        }
    }
    for (int i = 9; i ≥ 0; i--)
        printf("%d", res[i]);
    printf("\n");

    return 0;
}
```

0x24 计数技巧

等价替代

当我们需要计算一些带特殊条件的方案数时，可以用一些等价替代的方 法。具体来讲，我们构造一个双射（一一映射），将每一种原问题的方案 映射为新问题的一种方案，并使答案更容易计算。 常用的有捆绑法、插空法、隔板法等。

0x24.1 捆绑法

也称整体法，在计数时，当要求某些元素必须相邻时，先把他们看作一个整体，然后把所有整体都当成一个个元素和剩余元素一起考虑计数。要注意：整体内部也有顺序，也要乘上整体内部的排列数。

Example A

$ABCDE$ 五个人要排队，A 和 B 要相邻， C 和 D 要相邻，求一共有多少种排列方法。

Solution

首先将 AB, CD 分别看成一个整体，然后变成 3 个元素的排列问题， 方案数为 $3! = 6$，然后考虑 AB，CD 内部的相对顺序，共有 $A_2^2 + A_2^2 = 4$ 种情况， 最终根据乘法原理答案为 $6 \times 4 = 24$ 。

0x24.2 插空法

当要求若干元素两两不相邻时，可以先将其他元素放好，然后将这些元素插入空隙或者序列两端当中，进行计数。

Example B

$ABCDEFG$ 七个人要排队，ABC 三个人两两不相邻，求方案数。 先将剩下四个人排好，有 $4!=24$ 种方案，然后将 ABC 三个人插入 $4+1=5$ 个空当中，有$A_5^3=5\times4\times3=60$ 种方案，最终答案为 $24\times60=1440$ 。

Example C

一张节目表上有 3 个节目，保持其相对顺序不变，再加入 2 个新节目，有多少种方案?

Solution

捆绑法 + 插空法

对于这两个新节目我们可以分为两种情况：

1. 若两个新节目相邻，那么 3 个节目有 4 个空，再考虑内部顺序，总方案数为 $C_4^1\times A_2^2$ 。
2. 若两个节目不相邻，那么 2 个节目插入 4 个空中，方案数为 A_4^2

则总方案数即 $C_4^1\times A_2^2+A_4^2$ 。

0x24.3 隔板法

在解决若干相同元素分组问题时，若要求分为 m 组，其中每组至少一个元素，则可以转化为在排成一列的这些元素中插入 $m-1$ 个 “ 隔板 ” ，达到分组目的。

可将不可区分物品分配问题 / 不定方程整数解问题 转化为 隔板组合问题。

Example C

n 个小球放到 m 个盒子，要求每个盒子不为空。

Solution

我们可以将这 n 个小球摆成一排，从左往右依次插入 $m-1$ 块隔板，代表每 个盒子分到的部分，即每种插板的方案和原来的每一种分配方案是一一对 应的，$m-1$块隔板插入 $n-1$ 个空当中（因为要求非空所以不能插在最左和最右位置），所以总方案数为：C_{n-1}^{m-1} 。

Example D

n 个小球放到 m 个盒子，每个盒子可以为空。

Solution

上一题可以抽象为数学模型：求方程 $x_1+x_2+\cdots+x_m=n$ 的整数解，满足 $x_i\geq1$ 。非空即将上题的限制改成 $x_i\geq0$，我们可以设 m 个新变量 $y_i=x_i+1\geq1$，转化为求方程 $y_1+y_2+\cdots+y_m=n+m$ 的非零整数解，这样就转化为了上一题的模型，直接使用上题的结论，得到总方案数为：C_{n+m-1}^{m-1}

Example E

经典例题：方程 $x_1+x_2+x_3+x_4+x_5=21$ 有多少个解？其中 x_i 是非负整数。且满足下列条件：

首先原式根据隔板法显然答案为 C_{25}^4

1）$x_1\geq1$

由于 $x_1\geq1$ ，我们可以设 $y_1+1=x_1\geq1,y_1\geq0$。这样所有的解均为非负整数。

方程转化为：

$$y_1+1+x_2+x_3+x_4+x_5=21 \tag{6}$$

即：

$$y_1+x_2+x_3+x_4+x_5=20 \tag{7}$$

答案显然为 $C_{20+5-1}^{5-1}=C_{24}^4$

2）$x_i\geq 2,i=1,2,3,4,5$

设 $y_1+2=x_1,y_2+2=x_2,\cdots,y_5+2=x_5$

则原方程转换为：

$$y_1+y_2+y_3+y_4+y_5=11 \tag{8}$$

显然答案为 $C_{11+4-1}^{5-1}=C_{15}^4$ 。

3）$0\leq x_1\leq 10$

我们只需要用总方案数减去 $x_1\geq 11$ 的方案数即可。

$x_1\geq 11$ 的方案数显然为 C_{14}^4 。

则合法的方案数为： $C(25,4)-C(14,4)$ 。

Example F

方程 $x_1+x_2+x_3\leq 11$ 有多少个非负整数解？即 x_i 是非负整数。

Solution

方程可以转化为：

$$x_1+x_2+x_3+x_4=11 \tag{9}$$

答案显然为 C_{14}^3 。

Example G

方程 $x_1+x_2+x_3=13$ 有多少个解？其中 x_1,x_2,x_3 是小于 6 的非负整数。

Solution

首先不考虑限制条件，总方案数为 $C(15,2)$ 。

设 A 代表 x_1,x_2,x_3 为非负整数且 $x_1\geq 6$ 的解的集合

设 B 代表 x_1,x_2,x_3 为非负整数且 $x_2\geq 6$ 的解的集合

设 C 代表 x_1,x_2,x_3 为非负整数且 $x_3\geq 6$ 的解的集合

故满足条件的解的数量为：

$$\begin{aligned}&C(15,2)-|A\cup B\cup C|\\=&C(15,2)-\{|A|+|B|+|C|-|A\cap B|-|A\cap C|-|B\cap C|+|A\cap B\cap C|\}\\=&C(15,2)-\{3\times C(9,2)-3\times C(3,2)+0\}\end{aligned} \tag{10}$$

注：$|A\cap B\cap C|=0$ ，因为不可能出现 $x_1+x_2+x_3=13$ 且 x_1,x_2,x_3 均大于 6 的情况，因为 $18\geq 13$ 。

Example G

有多少种方式把 6 个玩具分给 3 个不同的孩子并使得每个孩子至少得到一个玩具。

Solution

首先不考虑限制条件，总方案数为 3^6 。

设 A 代表的第一个孩子没有得到玩具的方案数的集合

设 B 代表的第二个孩子没有得到玩具的方案数的集合

设 C 代表的第三个孩子没有得到玩具的方案数的集合

故满足条件的分配方式的数量为：

$$\begin{aligned} & 3^6-|A \cup B \cup C| \\ =& 3^6-\{|A|+|B|+|C|-|A \cap B|-|A \cap C|-|B \cap C|+|A \cap B \cap C|\} \\ =& 3^6-\{3 \times 2^6-3+0\} \\ =& 540 \end{aligned} \tag{11}$$

注：$|A \cap B \cap C|=0$ ，因为不可能出现的三个孩子均没有玩具的情况。

竞赛例题选讲

Problem A Character Encoding (hdu 6397)

给三个数 n、m、k， 在 $0 \sim n-1$ 这 n 个数中选出 m 个数排成一排，使得这 m 个数的和等于 k ，这 m 个数可以相同（可以重复选），只要排列不同即可。求一共有多少种排列方式是满足题意的。

Solution

首先考虑没有限制条件的情况，即任选 m 个数使得 m 个数的和为 k ，转换为数学模型即为：

$x_1+x_2+x_3+\cdots+x_m=k$

答案为该方程的非负整数解的个数。

显然根据隔板法解的方案数为 C_{k+m-1}^{m-1}

但是题目中限制了大小，即只能从全排列 $0 \sim n-1$ 中选择，即 $x_i<n$，所以我们需要使用容斥原理减掉不合法的方案数。

但是题目限制数的大小只能在 n 以内，那么可以这样考虑：

假设在我们的枚举中有 i 个数超出限制，即在 m 个数里，有 i 个大于等于 n，那么这时的方案数为：
$C_m^i \times C_{k+m-1-i \times n}^{m-1}$，即对应的方程为：
$x_1'+x_2'+\ldots+x_m'=k-i \times n$。
我们对于每一个i ，使用容斥原理，奇减偶加即可得出不合法的方案数，用总方案数减去不合法的方案数即为所求。

Code

```cpp
#include <cstdio>
#include <iostream>
#include <algorithm>
#include <cstring>
#include <cmath>
#include <map>
#include <queue>
using namespace std;
typedef long long ll;
typedef int itn;
#define int long long
typedef pair<int, int>PII;
const int N = 2e5 + 7, mod = 998244353;
const double PI = acos(-1.0);

```

```cpp
int n, m, k;
int fact[N], invfact[N];

int qpow(int a, int b)
{
    int res = 1;
    while(b) {
        if(b & 1) res = 1ll * res * a % mod;
        a = 1ll * a * a % mod;
        b >>= 1;
    }
    return res;
}

int inv(int x)
{
    return qpow(x, mod - 2);
}


int C(int n, int m)
{
    if(n < m) return 0;
    return ((1ll * fact[n] * invfact[m]) % mod * 1ll * invfact[n - m]) % mod;
}

void init(int n)
{
    fact[0] = fact[1] = 1;
    for(int i = 2; i ≤ n; ++ i) {
        fact[i] = (1ll * fact[i - 1] * i) % mod;
    }
    for(int i = 0; i ≤ n; ++ i) {
        invfact[i] = inv(fact[i]);
    }
}

signed main()
{
    init(N - 7);
    int t;
    scanf("%lld", &t);
    while(t -- ) {
        scanf("%lld%lld%lld", &n, &m, &k);
        int ans = C(k + m - 1, m - 1);

        int illegal = 0;
        for(int i = 1; i ≤ m; ++ i) {
            if(i & 1)
                illegal = (illegal + 1ll * C(m, i) * C(k + m - 1 - i * n, m - 1) % mod) % mod;
            else
                illegal = (illegal - 1ll * C(m, i) * C(k + m - 1 - i * n, m - 1) % mod + mod) %
mod;

        }
```

```
70
71          ans = ((ans - illegal) % mod + mod) % mod;
72          printf("%lld\n", ans);
73      }
74      return 0;
75  }
```

0x30 多重集

集合的概念是唯一性。多重集的特点就是不唯一性。也就是同一种元素可以在多重集里面多次出现。

设多重集合 $S=\{n_1\times a_1,n_2\times a_2,\cdots,n_k\times a_k\},N=n_1+n_2+\cdots+n_k$，

即集合 S 中含有 n_1 个元素 a_1， n_2 个元素 a_2，$\cdots$，n_k 个元素 a_k，n_i 被称为元素 a_i 的重数，k 成为多重集合的类别数。

0x31 多重集的排列数问题

在 S 中任选 r 个元素的排列称为 S 的 r 排列，当 $r=N$ 时，有公式

$$P(N;n_1\times a_1,n_2\times a_2,\cdots,n_k\times a_k)=\frac{N!}{n_1!\times n_2!\times\cdots\times n_k}$$

假设多重集一共有 N 个元素。那么对这 N 个元素全排列，除掉相同元素的全排列的积即可。其实就是先把所有可能，也就是全排列处理出来，然后相同元素可以随意互换位置，按乘法原理除掉就行。

- 设元素 $a_1,a_2,\ldots,a_n$ 互不相同，从无限多重集 $\{\infty\times a_1,\infty\times a_2,\ldots,\infty\times a_n\}$ 中取 r 个元素的排列数为 n^r。
- 设元素 $a_1,a_2,\ldots,a_n$ 互不相同，从有限多重集 $\{k_1\times a_1,k_2\times a_2,\ldots,k_n\times a_n\}$ 中取 r 个元素的排列数为 n^r，各 k 均大于等于 r 。
- 设元素 $a_1,a_2,\ldots,a_n$ 互不相同，从有限多重集 $\{k_1\times a_1,k_2\times a_2,\ldots,k_n\times a_n\}$ 中元素的全排列为 $\frac{(k_1+k_2+\ldots+k_n)!}{k_1!\times k_2!\times\ldots\times k_n!}$
- 设元素 $a_1,a_2,\ldots,a_n$ 互不相同，从有限多重集 $\{k_1\times a_1,k_2\times a_2,\ldots,k_n\times a_n\}$ 中取 r 个元素，至少存在一个 $k_i<r$ 时，排列数为 $r\times(\frac{r!}{k_1!\times k_2!\times\ldots\times k_n!})$，即指数型生成函数 $G(x)=1+\frac{x}{1!}+\frac{x^2}{2!}+\ldots+\frac{x_i^k}{k_i!}$ 中 x^r 的系数

0x32 多重集的组合数问题

在 S 中任选 r 个元素的组合称为 S 的 r 组合，当 $\forall i,r\le n_i$ 时，有公式

$$C(n;n_1\times a_1,n_2\times a_2,\cdots,n_k\times a_k)=C(k+r-1,r)$$

由公式可以看出多重集合的组合只与类别数 k 和选取的元素 r 有关，与总数无关。

多重集的组合

- 设元素 $a_1, a_2, \ldots, a_n$ 互不相同，从无限多重集 $\{\infty \times a_1, \infty \times a_2, \ldots, \infty \times a_n\}$ 中取 r 个元素的组合数为 $C(n+r-1, r)$
- 设元素 $a_1, a_2, \ldots, a_n$ 互不相同，从有限多重集 $\{k_1 \times a_1, k_2 \times a_2, \ldots, k_n \times a_n\}$，各 k 均大于等于 r 从中取 r 个元素的组合数为 $C(n+r-1, r)$
- 设元素 $a_1, a_2, \ldots, a_n$ 互不相同，从有限多重集 $\{k_1 \times a_1, k_2 \times a_2, \ldots, k_n \times a_n\}$ 中取r个元素，至少存在一个 $k_i < r$时，令生成函数$G(x) = 1 + x + x^2 + \ldots + x^{k_i}, i = 1, 2 \ldots n$，$G(x)$ 中 x^r 的系数即为所求。

0x40 二项式定理

0x41 一般二项式定理

n 为非负整数：

$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}$

0x42 扩展二项式定理

n 为非负整数：

$\left(\sum_{i=1}^{t} x_i\right)^n = \sum_{\sum_{i=1}^{t} n_i = n} \left(\binom{n}{n_1, n_2, \cdots, n_t} \prod_{j=1}^{t} x_j^{n_j} \right)$

0x43 广义二项式定理

n 为实数：

$(x+y)^n = \sum_k \frac{n^{\underline{k}}}{k!} x^k y^{n-k}$

0x50 特殊的数列

0x51 斐波那契数（Fibonacci）

斐波那契数列（Fibonacci sequence），又称黄金分割数列，因数学家莱昂纳多·斐波那契（Leonardoda Fibonacci）以兔子繁殖为例子而引入，故又称为“兔子数列”，指的是这样一个数列：0、1、1、2、3、5、8、13、21、34、……

http://oeis.org/A000045

递推公式：

$$fib_n = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ fib_{n-1} + fib_{n-2} & n > 1 \end{cases}$$

推导结论：

性质51.1：$\sum_{i=1}^{n} f_i = f_{n+2} - 1$

性质51.2：$\sum_{i=1}^{n} f_{2i-1} = f_{2n}$

性质51.3：$\sum_{i=1}^{n} f_{2i} = f_{2n+1} - 1$

性质51.4：$\sum_{i=1}^{n} (f_n)^2 = f_n f_{n+1}$

性质51.5：$f_{n+m} = f_{n-1} f_{m-1} + f_n f_m$

性质51.6：$(f_n)^2 = (-1)^{(n-1)} + f_{n-1} f_{n+1}$

性质51.7：$f_{2n-1} = (f_n)^2 - (f_{n-2})^2$

性质51.8： $f_n = \frac{f_{n+2}+f_{n-2}}{3}$

性质51.9： $\frac{f_i}{f_{i-1}} \approx \frac{\sqrt{5}-1}{2} \approx 0.618$

性质51.10： $f_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$ （证明）

生成函数： $F(x) = \frac{x}{1-x-x^2} = \frac{\left(\frac{1}{1-\frac{1+\sqrt{5}}{2}x} - \frac{1}{1-\frac{1-\sqrt{5}}{2}x}\right)}{\sqrt{5}}$

0x52 卡特兰数（Catalan）

0x52.1 卡特兰数

卡特兰数列是组合数学中一个常出现在各种计数问题中出现的数列，其前几项为：
$1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, \ldots\ldots$

卡特兰数首先是由欧拉在计算对凸 n 边形的不同的对角三角形剖分的个数问题时得到的，即在一个凸 n 边形中，通过不相交于 n 边形内部的对角线，把 n 边形拆分成若干三角形，不同的拆分数目用 H_n 表示，H_n 即为卡特兰数。

http://oeis.org/A000108

递归公式 1

$$f(n) = \sum_{i=0}^{n-1} f(i) * f(n-i-1)$$

递归公式 2

$$f(n) = \frac{f(n-1)*(4*n-2)}{(n+1)}$$

组合公式 1

$$f(n) = \frac{C_{2n}^n}{(n+1)}$$

组合公式 2

$$f(n) = C_{2n}^n - C_{2n}^{n-1}$$

通项公式

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{\binom{2n}{n}}{n+1} = \frac{C_{n-1}(4n-2)}{n+1}$$

生成函数

$$C(x) = \frac{1-\sqrt{1-4x}}{2x}$$

0x52.2 卡特兰数的应用

二叉树的计数： 已知二叉树有 n 个结点，求能构成多少种不同的二叉树。
括号化问题： 一个合法的表达式由 `()` 包围，`()` 可以嵌套和连接，如：`(())()` 也是合法表达式，现给出 n 对括号，求可以组成的合法表达式的个数。
划分问题： 将一个凸 $n+2$ 多边形区域分成三角形区域的方法数。
出栈问题： 一个栈的进栈序列为 $1, 2, 3, .. n$，求不同的出栈序列有多少种。
路径问题： 在 $n \times n$ 的方格地图中，从一个角到另外一个角，求不跨越对角线的路径数有多少种。
握手问题： $2n$ 个人均匀坐在一个圆桌边上，某个时刻所有人同时与另一个人握手，要求手之间不能交叉，求共有多少种握手方法。

0x52.3 求卡特兰数

求卡特兰数，$n \leq 35$ 。

```
LL h[36];
void init() {
    h[0] = h[1] = 1;
    for (int i = 2; i ≤ 35; i++) {
        h[i] = 0;
        for (int j = 0; j < i; j++)
            h[i] = h[i] + h[j] * h[i - j - 1];
        cout << h[i] << endl;
    }
}
```

求卡特兰数，$n < 100$ 。

```
#define BASE 10000
int a[100 + 5][100];
void multiply(int num, int n, int b) { //大数乘法
    int temp = 0;

    for (int i = n - 1; i ≥ 0; i--) {
        temp += b * a[num][i];
        a[num][i] = temp % BASE;
        temp /= BASE;
    }
}
void divide(int num, int n, int b) { //大数除法
    int div = 0;

    for (int i = 0; i < n; i++) {
        div = div * BASE + a[num][i];
        a[num][i] = div / b;
        div %= b;
    }
}
void init() {
    memset(a, 0, sizeof(a));
    a[1][100 - 1] = 1;

    for (int i = 2; i ≤ 100; i++) {
        memcpy(a[i], a[i - 1], sizeof(a[i - 1]));
        multiply(i, 100, 4 * i - 2);
        divide(i, 100, i + 1);
    }
}
int main() {
    init();
    int n;

    while (scanf("%d", &n) ≠ EOF) {
        int i;

        for (i = 0; i < 100 && a[n][i] == 0; i++);

```

```
        printf("%d", a[n][i++]);

        for (; i < 100; i++)
            printf("%04d", a[n][i]);

        printf("\n");
    }

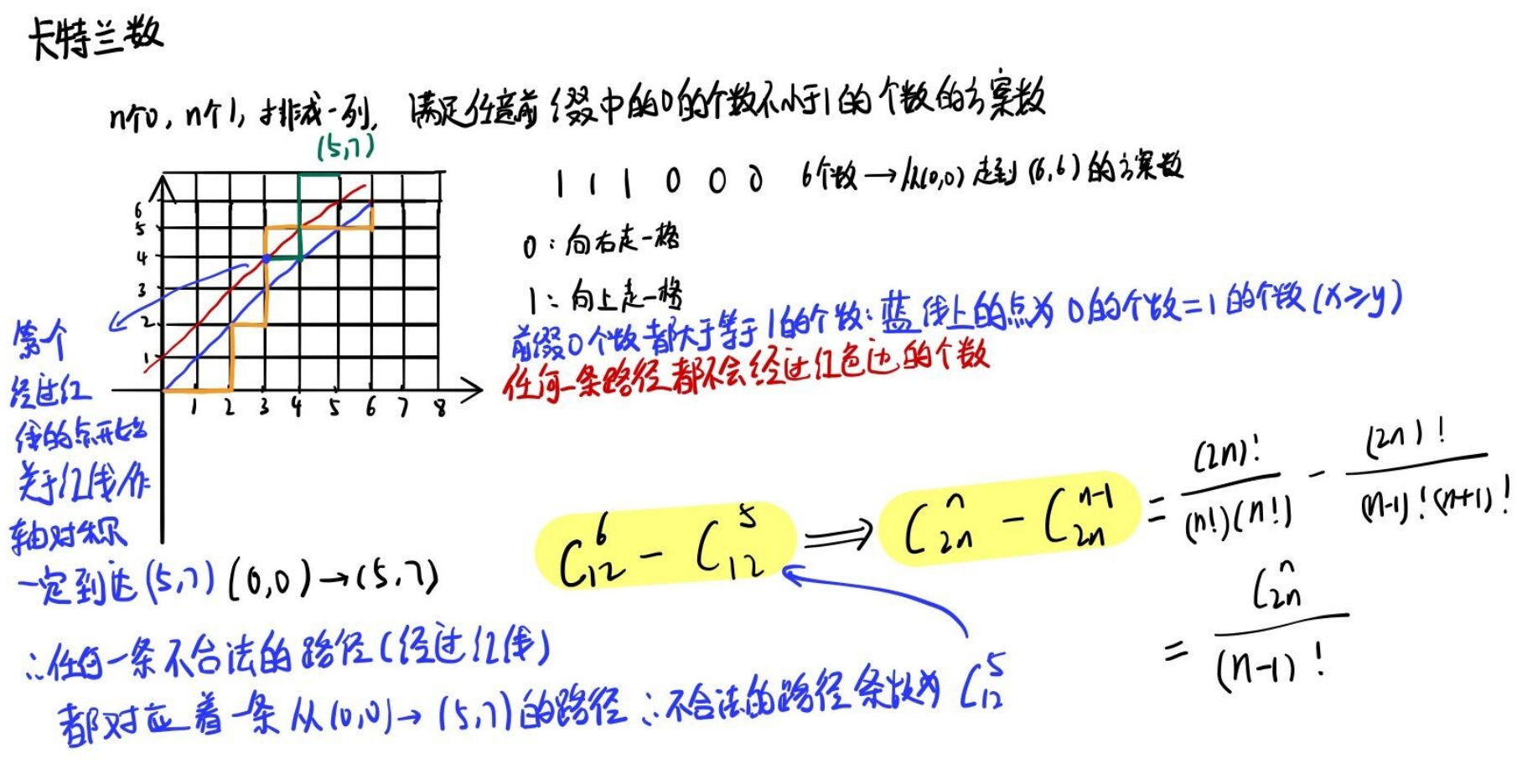
    return 0;
}
```

0x52.4 竞赛例题选讲

Problem A 满足条件的 01 序列（AcWing 889）

题目描述：给定 n 个 0 和 n 个 1，它们将按照某种顺序排成长度为 $2n$ 的序列，求它们能排列成的所有序列中，能够满足任意前缀序列中 0 的个数都不少于 1 的个数的序列有多少个。答案对 10^9+7 取模。

数据范围：$1 \le n \le 10^5$



https://blog.csdn.net/weixin_45697774

```
const int N = 200007, mod = 1e9 + 7;
const double eps = 1e-6;
typedef long long ll;

int n, m;
int fact[N], infact[N];

int qpow(int a, int b, int mod)
{
    int res = 1;
    while(b){
        if(b & 1)res = (ll)res * a % mod;
        a = (ll)a * a % mod;
        b >>= 1;
```

```
    }
    return res;
}

void init()
{
    fact[0] = infact[0] = 1;
    for(int i = 1; i < N; ++ i){
        fact[i] = (ll)fact[i - 1] * i % mod;
        infact[i] = (ll)infact[i - 1] * qpow(i, mod - 2, mod) % mod;
    }
}

int main()
{
    init();
    scanf("%d", &n);
    int res = (ll)fact[2 * n] * infact[n] % mod * infact[n] % mod * qpow(n + 1, mod - 2, mod) %
mod;
    printf("%d\n", res);
    return 0;
}
```

0x53 斯特林数（Stirling）

> 更多详细的斯特林数的内容详见：《小学生都能看懂的三类斯特林数从入门到升天教程 》（含性质完整证明、斯特林反演、拉赫数）

Stirling数出现在许多组合枚举问题中。对**第一类Stirling数** $s\left(n,m\right)$，也可记为 $\begin{bmatrix}n\\m\end{bmatrix}$。表示将 n 个不同元素构成 m 个圆排列的方案数。同时还分为**无符号第一类Stirling数** $s_u\left(n,m\right)$ 和**有符号第一类Stirling数** $s_s\left(n,m\right)$。**第二类Stirling数** $S\left(n,m\right)$，同时可记为 $\begin{Bmatrix}n\\m\end{Bmatrix}$，表示将 n 个不同的元素划分成 m 个集合的方案数。（第一类和第二类Stirling数在符号上，一个是小写 s，一个是大写 S）。拉赫数与斯特林数关系密切，所以有时拉赫数也被称为 **第三类斯特林数**，将在文末探讨它的定义和一些简单性质。

0x53.0 上升幂与下降幂

我们记 **上升阶乘幂（上升幂）**

$$x^{\overline{n}}=\prod_{k=0}^{n-1}(x+k)=x(x+1)(x+2)\cdots(x+n-1)$$

普通的幂就是 $x\times x\times x\times\cdots$，上升幂就可以形象地理解为 x 越乘越大，在往上升。

我们就可以利用下面的恒等式将上升幂转化为普通幂：

$$x^{\overline{n}}=\sum_{k=0}^{n}\begin{bmatrix}n\\k\end{bmatrix}x^k$$

如果将普通幂转化为上升幂，则有下面的恒等式：

$$x^n=\sum_k\begin{Bmatrix}n\\k\end{Bmatrix}(-1)^{n-k}x^{\overline{k}}$$

我们记 **下降阶乘幂（下降幂）**

$x^{\underline{n}}=\frac{x!}{(x-n)!}=\prod_{k=0}^{n-1}(x-k)$ 。
则可以利用下面的恒等式将普通幂转化为下降幂：

$$x^n=\sum_k \begin{Bmatrix} n \\ k \end{Bmatrix} x^{\underline{k}}$$

如果将下降幂转化为普通幂，则有下面的恒等式：

$$x^{\underline{n}}=\sum_k \begin{bmatrix} n \\ k \end{bmatrix} (-1)^{n-k}x^k$$

多项式下降阶乘幂表示与多项式点值表示的关系

在这里，多项式的下降阶乘幂表示就是用

$$f(x)=\sum_{i=0}^{n} b_i x^{\underline{i}}$$

的形式表示一个多项式，而点值表示就是用 $n+1$ 个点

$$(i,a_i),i=0\cdots n$$

来表示一个多项式。
显然，下降阶乘幂 b 和点值 a 间满足这样的关系：

$$a_k=\sum_{i=0}^{n} b_i k^{\underline{i}}$$

即

$$a_k=\sum_{i=0}^{n} \frac{b_i k!}{(k-i)!}$$

$$\frac{a_k}{k!}=\sum_{i=0}^{k} b_i \frac{1}{(k-i)!}$$

很明显这是一个卷积形式的式子，我们可以在 $\mathcal{O}(nlogn)$ 的时间复杂度内完成点值和下降阶乘幂的互相转化。

0x53.1 第一类斯特林数

定义 第一类斯特林数 为将 n 个元素，划分为 k 个圆排列的方案数，记作 $s(n,k)$，或 $\begin{bmatrix} n \\ k \end{bmatrix}$。又根据正负性分为无符号第一类斯特林数 $s_u\left(n,m\right)$ 和带符号第一类斯特林数 $s_s\left(n,m\right)$。组合数学中的第一类斯特林数一般指无符号的第一类斯特林数。

递推式：

$$\begin{bmatrix} n \\ k \end{bmatrix}=\begin{bmatrix} n-1 \\ k-1 \end{bmatrix}+(n-1)\times\begin{bmatrix} n-1 \\ k \end{bmatrix}$$

递推边界：

- $s(n,n)=1(n\geq 0)$
- $s(n,0)=0(n\geq 1)$

> 我们将在本文 **0x13第一类斯特林数的性质** 中证明上述边界性质。

Template A 第一类斯特林数（AcWing 3165）

给定 n 和 k，求第一类斯特林数 $s(n,k)\mod 1e9+7$ 。

时间复杂度为：$\mathcal{O}(n^2)$

Code

```cpp
const int N = 5007, mod = 1e9 + 7;
typedef long long ll;
typedef int itn;
int n, m, k;
ll s[N][N];

int main()
{
    scanf("%d%d", &n, &k);
    s[0][0] = 1;
    for(int i = 1; i ≤ n; ++ i) {
        for(int j = 1; j ≤ k; ++ j) {
            s[i][j] = (s[i - 1][j - 1] + (ll)(i - 1) * s[i - 1][j] % mod) % mod;
        }
    }
    printf("%lld\n", s[n][k]);
    return 0;
}
```

二项式系数 $C\left(n,m\right)$ 可以构成一个杨辉三角（pascal三角形）。同样第一类斯特林数同样也可以构成一个三角如图13.1 所示，我们可以由此分析其性质。

	无符号Stirling数	有符号Stirling数
n=0	1	1
n=1	0 1	0 1
n=2	0 1 1	0 -1 1
n=3	0 2 3 1	0 2 -3 1
n=4	0 6 11 6 1	0 -6 11 -6 1
n=5	0 24 50 35 10 1	0 24 -50 35 -10 1
n=6	0 120 274 225 85 15 1	0 -120 274 -225 85 -15 1
n=7	0 720 1764 1624 735 175 21 1	0 720 -1764 1624 -735 175 -21 1

图13.1

▲ 无符号第一类斯特林数性质

性质53.1.1： $s_u\left(0,0\right)=1$

性质53.1.2： $s_u\left(n,0\right)=0$

性质53.1.3： $s_u\left(n,n\right)=1$

性质53.1.4： $s_u\left(n,1\right)=(n-1)!$

性质53.1.5： $s_u\left(n,n-1\right)=\mathrm{C}_n^2$

性质53.1.6： $s_u\left(n,2\right)=(n-1)!\cdot\sum\limits_{i=1}^{n-1}\frac{1}{i}$

性质53.1.7： $s_u\left(n,n-2\right)=2\cdot\mathrm{C}_n^3+3\cdot\mathrm{C}_n^4$

性质53.1.8： $\sum\limits_{k=0}^{n}s_u\left(n,k\right)=n!$

有无符号斯特林数分别表现为其升阶函数和降阶函数的各项系数（类似于二项式系数），形式如下：

有符号斯特林数的生成函数为降阶函数（下降幂）：

$x^{\underline{n}}=x^{n\downarrow}=x\left(x-1\right)\left(x-2\right)\cdots\left(x-n+1\right)=\sum_{k=0}^{n}s_s\left(n,k\right)x^k$

无符号斯特林数的的生成函数为升阶函数（上升幂）：

$$x^{\bar{n}}=x^{n\uparrow}=x\left(x+1\right)\left(x+2\right)\cdots\left(x+n-1\right)=\sum_{k=0}^{n}s_u\left(n,k\right)x^k$$

Problem A 第一类斯特林数•行 （ luogu P5408 ）

第一类斯特林数 $\begin{bmatrix}n\\m\end{bmatrix}$ 表示将 n 个不同元素构成 m 个圆排列的数目。

给定 n ，对于所有的整数 $i\in[0,n]$ ，你要求出 $\begin{bmatrix}n\\i\end{bmatrix}$ 。

由于答案会非常大，所以你的输出需要对 167772161 （$2^{25}\times5+1$ ，是一个质数）取模。

输入格式
一行一个正整数 n ，意义见题目描述。

输出格式
共一行 $n+1$ 个非负整数。

你需要按顺序输出 $\begin{bmatrix}n\\0\end{bmatrix},\begin{bmatrix}n\\1\end{bmatrix},\begin{bmatrix}n\\2\end{bmatrix},\ldots,\begin{bmatrix}n\\n\end{bmatrix}$ 的值。

Solution

我们知道无符号第一类斯特林数的生成函数为上升幂：

$$x^{\bar{n}}=x^{n\uparrow}=x\left(x+1\right)\left(x+2\right)\cdots\left(x+n-1\right)=\sum_{k=0}^{n}s_u\left(n,k\right)x^k$$

也就是说我们只需要求出上升幂 $x^{\bar{n}}$ 即可推出第一类斯特林数的值。

很明显我们可以使用倍增法，利用多项式来求解。

显然有

$$x^{\bar{2n}}=x^{\bar{n}}(x+n)^{\bar{n}}$$

也就是

$$\prod_{i=0}^{2n-1}(x+i)=\prod_{i=0}^{n-1}(x+i)\prod_{i=0}^{n-1}(x+n+i)$$

（展开后等式两边相等）

假设我们已经知道了 $x^{\bar{n}}$ ，那么瓶颈就在于如何算出 $(x+n)^{\bar{n}}$

设 $f(x)=x^{\bar{n}}$，那么 $(x+n)^{\bar{n}}$ 就是 $f(x+n)$

那么问题就变为怎么从多项式 $f(x)$ 推到多项式 $f(x+n)$，也就是说我们知道了一个 n 次多项式 $f(x)$，如何快速计算出 $f(x+n)$ 。

假设 $f(x)$ 的第 i 项系数为 a_i ，即 $a^i=[x^i]f(x)$

则：

$$\begin{aligned}f(x+n)&=\sum_{i=0}^{n}a_i(x+n)^i\\&=\sum_{i=0}^{n}a_i\sum_{j=0}^{i}x^jn^{i-j}\binom{i}{j}\\&=\sum_{j=0}^{n}x^j\sum_{i=j}^{n}a_in^{i-j}\binom{i}{j}\\&=\sum_{j=0}^{n}\frac{x^j}{j!}\sum_{i=j}^{n}(a_ii!)\frac{n^{i-j}}{(i-j)!}\end{aligned}$$

我们设多项式 $A(i)=a_ii!,B(i)=\frac{n^i}{i!}$ ，就有：

$$\begin{aligned}f(x+n)&=\sum_{j=0}^{n}\frac{x^j}{j!}\sum_{i=j}^{n}A(i)\times B(i-j)\\&=\sum_{j=0}^{n}\frac{x^j}{j!}\sum_{i=0}^{n-j}A(i+j)\times B(i)\\&=\sum_{j=0}^{n}\frac{x^j}{j!}\sum_{i=0}^{n-j}A'(n-j-i)\times B(i)\end{aligned}$$

其中多项式 A' 表示 A 反转后的多项式。

很明显这里就是两个多项式的卷积，我们可以直接使用多项式乘法求解，$\mathcal{O}(n\log n)$ 的时间复杂度求出 $f(x+n)$ 。

这样我们就得到了快速求得 $f(x+n)$ 的方法，最后我们只需要再用一次多项式乘法求一下 $f(x)f(x+n)$ 即可。

总时间复杂度使用倍增，和FFT的时间复杂度一样， $T(n)=T(\frac{n}{2})+\mathcal{O}(n\log n)=\mathcal{O}(n\log n)$

Code

```cpp
#include <cstdio>
#include <iostream>
#include <algorithm>
#include <cstring>
#include <cmath>

using namespace std;
#define int long long
typedef long long ll;
const ll mod = 167772161;
ll G = 3, invG;
const int N = 1200000;
ll qpow(ll b, int n) {
    ll res = 1;
    while (n) {
        if (n & 1)
            res = res * b % mod;
        b = b * b % mod;
        n >>= 1;
    }
    return res;
}
int read() {
    int x = 0;
    char ch = getchar();
    while (!isdigit(ch))
        ch = getchar();
    while (isdigit(ch))
        x = (x * 10 + (ch - '0')) % mod, ch = getchar();
    return x;
}
int R[N];
void NTT(ll *f, int n, int fl) {
    for (int i = 0; i < n; ++i)
        if (i < R[i])
            swap(f[i], f[R[i]]);

```

```
    for (int p = 2; p ≤ n; p <<= 1) {
        int len = (p >> 1);
        ll w = qpow((fl == 0) ? G : invG, (mod - 1) / p);

        for (int st = 0; st < n; st += p) {
            ll buf = 1, tmp;
            for (int i = st; i < st + len; ++i) {
                tmp = buf * f[i + len] % mod;
                f[i + len] = (f[i] - tmp + mod) % mod;
                f[i] = (f[i] + tmp) % mod;
                buf = buf * w % mod;
            }
        }
    }
    if (fl == 1) {
        ll invN = qpow(n, mod - 2);
        for (int i = 0; i < n; ++i)
            f[i] = (f[i] * invN) % mod;
    }
}
void Mul(ll *f, ll *g, int n, int m) {
    m += n;
    n = 1;
    while (n < m)
        n <<= 1;
    for (int i = 0; i < n; ++i)
        R[i] = (R[i >> 1] >> 1) | ((i & 1) ? (n >> 1) : 0);
    NTT(f, n, 0);
    NTT(g, n, 0);
    for (int i = 0; i < n; ++i)
        f[i] = f[i] * g[i] % mod;
    NTT(f, n, 1);
}
ll inv[N], fac[N];
ll w[N], a[N], b[N], g[N];
void Solve(ll *f, int m) {
    if (m == 1)
        return f[1] = 1, void(0);
    if (m & 1) {
        Solve(f, m - 1);
        for (int i = m; i ≥ 1; --i)
            f[i] = (f[i - 1] + f[i] * (m - 1) % mod) % mod;
        f[0] = f[0] * (m - 1) % mod;
    } else {
        int n = m / 2;
        ll res = 1;
        Solve(f, n);
        for (int i = 0; i ≤ n; ++i)
            a[i] = f[i] * fac[i] % mod, b[i] = res * inv[i] % mod, res = res * n % mod;
        reverse(a, a + n + 1);
        Mul(a, b, n + 1, n + 1);
        for (int i = 0; i ≤ n; ++i)
            g[i] = inv[i] * a[n - i] % mod;
        Mul(f, g, n + 1, n + 1);
        int limit = 1;
```

```
        while (limit < (n + 1) << 1)
            limit <<= 1;
        for (int i = n + 1; i < limit; ++i)
            a[i] = b[i] = g[i] = 0;
        for (int i = m + 1; i < limit; ++i)
            f[i] = 0;
    }
}
ll f[N];
void init(int n) {
    fac[0] = 1;
    for (int i = 1; i ≤ n; ++i)
        fac[i] = 1ll * fac[i - 1] * i % mod;
    inv[n] = qpow(fac[n], mod - 2);
    for (int i = n - 1; i ≥ 0; --i)
        inv[i] = 1ll * inv[i + 1] * (i + 1) % mod;
}
signed main() {
    invG = qpow(G, mod - 2);
    int n, k = 0;
    cin >> n;
    init(n + n);
    Solve(f, n);
    for (int i = 0; i ≤ n; ++i)
        printf("%lld ", f[i]);

    return 0;
}
```

Problem B 第一类斯特林数•列 （ luogu P5409 ）

第一类斯特林数 $\begin{bmatrix} n \\ m \end{bmatrix}$ 表示将 n 个不同元素构成 m 个圆排列的数目。

给定 n,k ，对于所有的整数 $i \in [0,n]$ ，你要求出 $\begin{bmatrix} i \\ k \end{bmatrix}$ 由于答案会非常大，所以你的输出需要对 167772161 （$2^{25} \times 5 + 1$，是一个质数）取模。

输入格式
一行两个正整数 n,k ，意义见题目描述。

输出格式
共一行 $n+1$ 个非负整数。

你需要按顺序输出 $\begin{bmatrix} 0 \\ k \end{bmatrix}, \begin{bmatrix} 1 \\ k \end{bmatrix}, \begin{bmatrix} 2 \\ k \end{bmatrix}, \ldots, \begin{bmatrix} n \\ k \end{bmatrix}$ 的值。

Solution

我们可以用指数型生成函数解决该问题。注意，由于递推公式和行有关，我们不能利用递推公式计算同列的第一类斯特林数。

显然，单个轮换的指数型生成函数为

$$F(x)=\sum_{i=1}^{n}\frac{(i-1)!x^i}{i!}=\sum_{i=1}^{n}\frac{x^i}{i}$$

它的 k 次幂就是 $\begin{bmatrix} i \\ k \end{bmatrix}$ 的指数型生成函数。直接计算即可。

时间复杂度：$\mathcal{O}(nlogn)$

```cpp
int main() {
    scanf("%d%d", &n, &k);
    fact[0] = 1;
    for (int i = 1; i ≤ n; ++i)
        fact[i] = (ll)fact[i - 1] * i % mod;
    ifact[n] = qpow(fact[n], mod - 2);
    for (int i = n - 1; i ≥ 0; -- i)
        ifact[i] = (ll)ifact[i + 1] * (i + 1) % mod;
    poly f(n + 1);
    for (int i = 1; i ≤ n; ++ i)
        f[i] = (ll)fact[i - 1] * ifact[i] % mod;
    f = poly_exp(log(f >> 1) * k) << k, f.resize(n + 1);
    for (int i = 0; i ≤ n; ++ i)
        printf("%lld ", 1ll * f[i] * fact[i] % mod * ifact[k] % mod);
    return 0;
}
```

0x53.2 第二类斯特林数

第二类斯特林数实际上是集合的一个拆分，表示将 n 个不同的元素拆分成 m 个集合间有序（可以理解为集合上有编号且集合不能为空）的方案数，记为 $S\left(n,m\right)$（这里是大写的）或者 $\begin{Bmatrix} n \\ m \end{Bmatrix}$ 。和第一类斯特林数不同的是，这里的集合内部是不考虑次序的，而圆排列圆的内部是有序的。常常用于解决组合数学中的几类放球模型。描述为：将 n 个不同的球放入 m 个无差别的盒子中，要求盒子**非空**，有几种方案？

第二类斯特林数要求盒子是无区别的，所以可以得到其方案数公式：

$$S\left(n,m\right)=\frac{1}{m!}\sum_{k=0}^{m}\left(-1\right)^{k}\binom{m}{k}\left(m-k\right)^{n}$$

递推公式：

$$\begin{Bmatrix} n \\ k \end{Bmatrix}=\begin{Bmatrix} n-1 \\ k-1 \end{Bmatrix}+k\times\begin{Bmatrix} n-1 \\ k \end{Bmatrix}$$

边界条件：

$$S(n,n)=1,p\geq 0$$

$$S(n,0)=0,n\geq 1$$

Template A 第二类斯特林数（AcWing 3166）

给定 n 和 k，求第二类斯特林数 $S(n,k)\mod 1e9+7$ 。

时间复杂度为：$\mathcal{O}(n^2)$

```cpp
typedef long long ll;
typedef int itn;
const int N = 5007, mod = 1e9 + 7;
int n, m, k;
int S[N][N];
int main()
{
    scanf("%d%d", &n, &k);
    S[0][0] = 1;
```

```
    S[n][0] = 0;
    for(int i = 1; i ≤ n; ++ i) {
        for(int j = 1; j ≤ k; ++ j) {
            S[i][j] = (S[i- 1][j - 1] + 1ll * j * S[i - 1][j]) % mod;//公式中的 k 是当前的 k
        }
    }
    cout << S[n][k] << endl;
    return 0;
}
```

n	第二类斯特林数
n=0	1
n=1	0 1
n=2	0 1 1
n=3	0 1 3 1
n=4	0 1 7 6 1
n=5	0 1 15 25 10 1
n=6	0 1 31 90 65 15 1
n=7	0 1 63 301 350 140 21 1
n=8	0 1 127 966 1701 1050 266 28 1
n=9	0 1 255 3025 7770 6951 2646 462 36 1

▲ 第二类斯特林数性质

性质53.1： $S\left(n,0\right)=0^n$

性质53.2： $S\left(n,1\right)=1$

性质53.3： $S\left(n,n\right)=1$

性质53.4： $S\left(n,2\right)=2^{n-1}-1$

性质53.5： $S\left(n,n-1\right)=C(n,2)$

性质53.6： $S\left(n,n-2\right)=C(n,3)+3\cdot C(n,4)$

性质53.7： $S\left(n,3\right)=\frac{1}{2}(3^{n-1}+1)-2^{n-1}$

性质53.8： $S\left(n,n-3\right)=C(n,4)+10\cdot C(n,5)+15\cdot C(n,6)$

性质53.9： $\sum\limits_{k=0}^{n}S(n,k)=B_n$，其中 B_n 是贝尔数

推论53.10： 若 $n<m$，$\sum\limits_{k=0}^{m}(-1)^k\binom{m}{k}(m-k)^n=0$

推论53.11： 因为 $S(m,m)=1$，所以有 $\sum\limits_{k=0}^{m}(-1)^k\binom{m}{k}(m-k)^m=m!$

推论53.12： $n^k=\sum\limits_{i=0}^{k}S(k,i)\times i!\times C_n^i$

普通型生成函数

$$\mathbf{G}_0(S(n,k))=\sum_{n\geqslant k}S(n,k)x^n=\frac{x^k}{(1-x)(1-2x)\cdots(1-kx)}=\frac{x^k}{\prod_{r=1}^{k}(1-rx)}$$

指数型生成函数

$\mathbf{G}_e(S(n,k))=\sum_{n\geqslant k}S(n,k)\cdot\frac{x^n}{n!}=\frac{(e^x-1)^k}{k!}$

第二类斯特林数的通项公式

$$\left\{ {n \atop m} \right\}=\sum_{i=0}^{m}\frac{(-1)^{m-i}i^n}{i!(m-i)!}$$

Problem A 第二类斯特林数·行 （ luogu P5395 ）

第二类斯特林数$\left\{ {n \atop m} \right\}$表示把 n 个不同元素划分成 m 个相同的集合中（不能有空集）的方案数。

给定 n ，对于所有的整数 $i\in[0,n]$ ，你要求出$\left\{ {n \atop i} \right\}$由于答案会非常大，所以你的输出需要对 167772161（$2^{25}\times5+1$，是一个质数）取模。

输入格式

一行一个正整数 n，意义见题目描述。

输出格式

共一行 $n+1$ 个非负整数。

你需要按顺序输出$\left\{ {n \atop 0} \right\},\left\{ {n \atop 1} \right\},\left\{ {n \atop 2} \right\},\ldots,\left\{ {n \atop n} \right\}$的值。

Solution

“同一行”的第二类斯特林数指的是，有着不同的 i ，相同的 n 的一系列 $\left\{ {n \atop i} \right\}$ 。求出同一行的所有第二类斯特林数，就是对 $i=0\cdots n$求出了将 n 个不同元素划分为 i 个非空集的方案数。

```cpp
int main() {
    scanf("%d", &n);
    fact[0] = 1;
    for (int i = 1; i ≤ n; ++i)
        fact[i] = 1ll * fact[i - 1] * i % mod;
    exgcd(fact[n], mod, ifact[n], ifact[0]),
          ifact[n] = (ifact[n] % mod + mod) % mod;
    for (int i = n - 1; i ≥ 0; --i)
        ifact[i] = 1ll * ifact[i + 1] * (i + 1) % mod;
    poly f(n + 1), g(n + 1);
    for (int i = 0; i ≤ n; ++i)
        g[i] = (i & 1 ? mod - 1ll : 1ll) * ifact[i] % mod,
               f[i] = 1ll * qpow(i, n) * ifact[i] % mod;
    f *= g;
    f.resize(n + 1);
    for (int i = 0; i ≤ n; ++i)
        printf("%d ", f[i]);
    return 0;
}
```

0x53.3 第三类斯特林数

拉赫数是由伊沃·拉赫在1954年发现的，因为拉赫数与斯特林数关系密切，所以有时拉赫数也被称为'''第三类斯特林数'''。可以用升阶函数或降阶函数定义为：

$$x^{\bar{n}}=\sum_{k=0}^{n}L(n,k)x^{\underline{k}}$$

或

$$x^{\underline{n}}=\sum_{k=0}^{n}(-1)^{n-k}L(n,k)x^{\bar{k}}$$

其中， $L(n,k)$ 即为拉赫数。
递推求第三类斯特林数

无符号拉赫数有如下递推关系：

$$L(n,k+1)=\frac{n-k}{k(k+1)}L(n,k)$$

或者：

$$L(n+1,k)=(n+k)L(n,k)+L(n,k-1)$$

边界条件：

$$L(n,0)=0$$

$$L(n,0)=0$$

$$L(n,k)=0\quad(\ k>n\quad)$$

$$L(1,1)=1$$

无符号拉赫数表

n \ k	0	1	2	3	4	5	6
0	1						
1	0	1					
2	0	2	1				
3	0	6	6	1			
4	0	24	36	12	1		
5	0	120	240	120	20	1	
6	0	720	1800	1200	300	30	1

第三类斯特林数（拉赫数）的简单性质

性质53.3.1： $L(0,0)=1$， $L(n,0)=0$ （ $n>0$ ）

性质53.3.2： 如果 $k>n$，有 $L(0,0)=1$， $L(n,k)=0$

性质53.3.3： $L(n,1)=n!$

性质53.3.4： $L(n,2)=\frac{(n-1)n!}{2}$

性质53.3.5： $L(n,3)=\frac{(n-2)(n-1)n!}{12}$

性质53.3.6： $L(n,n-1)=n(n-1)$

性质53.3.7： $L(n,n)=1$

性质53.3.8： $\sum_{n\geq k}L(n,k)\frac{x^n}{n!}=\frac{1}{k!}\left(\frac{x}{1-x}\right)^k$

无符号拉赫数计算公式可以作进一步拓展：

$$L(n,k)=\binom{n-1}{k-1}\frac{n!}{k!}=\binom{n}{k}\frac{(n-1)!}{(k-1)!}=\binom{n}{k}\binom{n-1}{k-1}(n-k)!$$

$$L(n,k)=\frac{n!(n-1)!}{k!(k-1)!}\cdot\frac{1}{(n-k)!}=\left(\frac{n!}{k!}\right)^2\frac{k}{n(n-k)!}$$

0x53.4 三类斯特林数之间的转换

两类斯特林数之间的递推式和实际含义很类似，他们之间存在一个互为转置的转化关系：

$\sum_{k=0}^{n} S1(n,k)S2(k,m) = \sum_{k=0}^{n} S2(n,k)S1(k,m)$

三类斯特林数以及乘方、阶乘之间的关系可以用 图40.1 表示

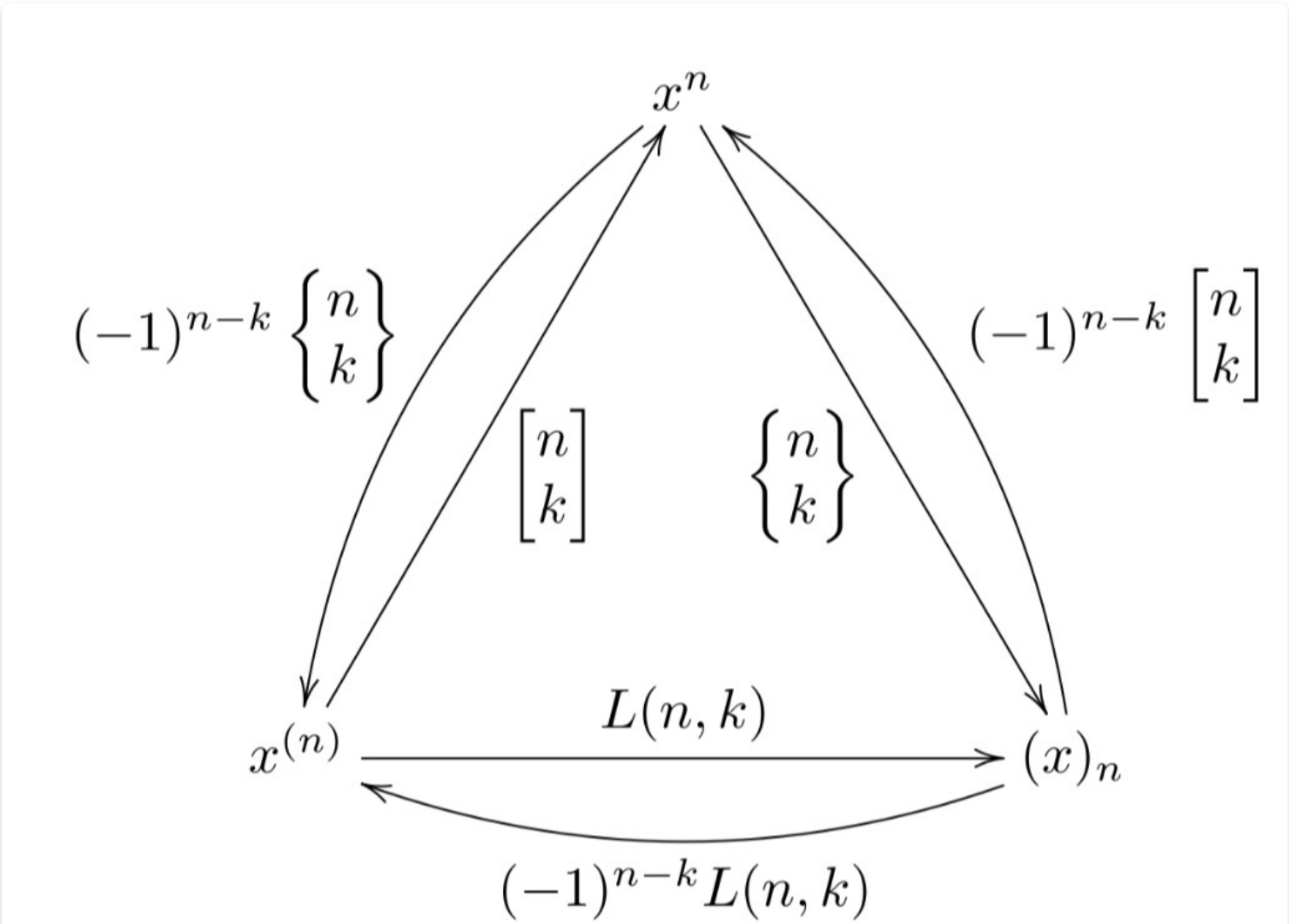


图40.1

无符号拉赫数与两类斯特林数都有关系 ， 关系如下：

$$L(n,k) = \sum_{j=0}^{n} \begin{bmatrix} n \\ j \end{bmatrix} \begin{Bmatrix} j \\ k \end{Bmatrix}$$

由无符号拉赫数与两类斯特林数之间的关系，考虑到两类斯特林数之间的关系，有

$\sum_{j\geq 0} L(n,j)L(j,k) = \delta_{nk}$

其中， δ_{nk} 是 克罗内克尔δ

0x52.5 斯特林反演

斯特林反演的公式：

$f(n) = \sum_{i=0}^{n} S(n,i)g(i) \Longleftrightarrow g(n) = \sum_{i=0}^{n}(-1)^{n-i}s(n,i)f(i)$

0x54 贝尔数（Bell）

0x54.1 贝尔数

意义：

B_n 表示将 n 个集合划分成若干个非空集合的方案数。

递推公式：

$B_n=\sum_{i=0}^{n-1}\binom{n-1}{i}B_i$

通项公式：

$B_n=\frac{1}{e}\sum_{i\geq 0}\frac{i^n}{i!}$

性质：

性质54.1： $B_n=\sum\limits_{i=0}^{n}S_2(n,i)$

性质54.2： 对于质数 p，$B_{p^m+n}=mB_n+B_{n+1}$

性质54.3： Bell 数列模质数 p 意义下循环节长度为 $\frac{p^p-1}{p-1}$

生成函数：$B(x)=e^{e^x-1}$

0x54.2 Bell 三角

- $b_{0,0}=1$
- 对于 $n>0$，$a_{n,1}=a_{n-1,n-1}$
- 对于 $n,m>0$，$a_{n,m}=a_{n,m-1}+a_{n-1,m-1}$
- $B_n=b_{n,0}$

0x55 分拆数

A000041

LOJ6268

意义： 将 n 进行整数分拆的方案数。

递推公式：

$$F_n=\begin{cases}0 & n<0\\ 1 & n=0\\ \sum\limits_{k\geq 1}-(-1)^k(F_{n-\frac{k(3k-1)}{2}}+F_{n-\frac{k(3k+1)}{2}}) & n>0\end{cases}$$

按照递推公式求分拆数的时间复杂度：$O(n\sqrt{n})$。

生成函数：

$$F(x)=\prod_{n>0}\frac{1}{1-x^n}=\frac{1}{\sum\limits_{k\geq 1}(-1)^k(x^{k(3k-1)}+x^{k(3k+1)})}$$

0x56 伯努利数（Bernoulli）

Bernoulli 数

设**等幂和** $S_m(n)=\sum_{k=1}^{n}k^m$，有公式：

$$S_m(n)=\frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k^+n^{m+1-k}$$

其中 B_k^+ 为 Bernoulli 数，B_n^+ 与 B_n^- 为两种不同的定义，仅在 $n=1$ 时有区别：$B_1^+=\frac{1}{2}$，$B_1^-=-\frac{1}{2}$。

递推公式：

$$\sum_{k=0}^{m}\binom{m+1}{k}B_k^-=0$$

边界为 $B_0^-=1$。

指数生成函数

$$B(x)=\frac{x}{e^x-1}$$

0x60 生成函数

> 生成函数详见：《小学生都能看懂的生成函数从入门到升天教程》《生成函数全家桶》

任意给定一个无限长的序列 $a_0,a_1,a_2,\cdots,a_n\cdots$

定义 函数 $g(x)=a_0x^0+a_1x^1+a_2x^2\cdots+a_nx^n+\cdots$

这是一个无穷级数，我们一般设 x 的取值范围为：$-1<x<1$，因为显然在这个范围内，无穷级数收敛，我们就可以在 $x\to\infty$ 的时候得到该无穷级数的值。

我们称这个函数 $g(x)$ 为序列的生成函数。实际上就是将方案数的序列映射称一个多项式的系数，将计数问题转化为多项式问题，这样我们就可以使用积累了很多年的解决方法远多于组合数学的多项式 / 无穷级数的方法，来求解这个问题。例如我们得到生成函数以后，我们就可以使用求极限，泰勒展开，求导积分等方法。

生成函数（generating function），又称母函数，是一种形式幂级数，其每一项的系数可以提供关于这个序列的信息。
生成函数有许多不同的种类，但大多可以表示为单一的形式：

$$F(x)=\sum_n a_nk^n(x)$$

其中 $k_n(x)$ 被称为核函数。不同的核函数会导出不同的生成函数，拥有不同的性质。举个例子：
普通生成函数：$k_n(x)=x^n$。
指数生成函数：$k_n(x)=\frac{x^n}{n!}$。
狄利克雷生成函数：$k_n(x)=\frac{1}{n^x}$。
另外，对于生成函数 $F(x)$，我们用 $[k_n(x)]F(x)$ 来表示它的第 n 项的核函数对应的系数，也就是 a_n。

0x61 普通生成函数

序列 a 的普通生成函数（ordinary generating function，OGF）定义为形式幂级数：

$$F(x)=\sum_n a_nx^n$$

其中 a 既可以是有穷序列，也可以是无穷序列。常见的几个例子（假设 a 以 0 为起点，也就是 x^0 对应组合数学里就是啥都不选的方案）

- 序列 $a=\langle1,2,3\rangle$ 的普通生成函数是 $1+2x+3x^2$ 。
- 序列 $a=\langle1,1,1,\cdots\rangle$ 的普通生成函数是 $\sum_{n\geq0}x^n$。
- 序列 $a=\langle1,2,4,8,16,\cdots\rangle$ 的生成函数是 $\sum_{n\geq0}2^nx^n$。
- 序列 $a=\langle1,3,5,7,9,\cdots\rangle$的生成函数是 $\sum_{n\geq0}(2n+1)x^n$ 。

换句话说，如果序列 a 有通项公式，那么它的普通生成函数的系数就是通项公式。

0x62 指数生成函数

对于序列 $\{a_0,a_1,a_2,\dots\}$，函数 $G_e(x)=a_0+\frac{a_1}{1!}x+\frac{a_2}{2!}x^2+\dots+\frac{a_k}{k!}x^k+\dots$称为序列 $\{a_0,a_1,a_2,\dots\}$对应的指数母函数。

指数型母函数可以理解为：对于 $\frac{x^i}{j!}$ 表示在一个方案中某个元素出现了 j 次，而在不同的位置中的 j 次出现是相同的，所以在排列计算总数时，只应算作一次，由排列组合的知识知道，最后的结果应该除以 $j!$

指数型母函数在使用过程中，一般会用到高等数学中的 e^x 的泰勒展开式：

$e^x=\sum_{n=0}^{\infty}\frac{x^n}{n!}=1+x+\frac{x^2}{2!}+\frac{x^3}{3!}+\dots+\frac{x^n}{n!}+\dots$

因此，指数型母函数可以简化为：$G_e(x)=b_0+b_1*\frac{x}{1!}+b_2*\frac{x^2}{2!}+b_3*\frac{x^3}{3!}+\dots$
其中，b_i 是所需的结果。

应用

指数型母函数常用于求多重排列数，即：有 n 种物品，已知每种物品的数量为 $k_1,k_2,\dots,k_n$ 个，求从中选出 m 件物品的排列数。

对于 n 个元素，其中 $a_0,a_1,a_2,\dots,a_n$ 互不相同，进行全排列，可得 $n!$ 个不同的排列。

若其中某个元素 a_i 重复了 n_i 次，那么全排列出来的必然有重复元素，而其中真正不同的排列数应为：$\frac{n!}{n_i!}$，即重复度为 $ni!$

同理，a_1 重复了 n_1 次，a_2 重复了 n_2 次，$\dots$，a_k 重复了 n_k 次 $(n_1+n_2+\dots+n_k=n)$

因此，对这样的 n 个元素进行全排列，可得到不同的排列个数实际上为：$\frac{n!}{n_1!n_2!\dots n_k!}$

若只对其中的 r 个元素进行全排列，就用到了指数型母函数：

构造母函数：$G(x)=(1+\frac{x}{1!}+\frac{x^2}{2!}+\dots+\frac{x_{k_1}}{k_1!})(1+\frac{x}{1!}+\frac{x^2}{2!}+\dots+\frac{x_{k_2}}{k_2!})\dots(1+\frac{x}{1!}+\frac{x^2}{2!}+\dots+\frac{x_{k_n}}{k_n!})$

化简得：$G(x)=a_0+a_1x+\frac{a_2}{2!}x^2+\frac{a_3}{3!}x^3+\dots+\frac{a_p}{p!}x^p$

其中 $p=k_1+k_2+\dots+k_n$，a_i为选出 i 个物品的排列方法数

注：若题中有限定条件，只需将第 i 项出现的列在第 i 项的式子中，未出现的不用列入

例如：物品 i 出现的次数为非 0 偶数，则原式改为：$\dots*(\frac{x^2}{2!}+\frac{x^4}{4!}+\dots+\frac{x^{ki}}{k_i!})*\dots$

0x70 群论

> 更多群论详细内容、证明、例题，详见：《小学生都能看懂的群论从入门到升天教程》《群论全家桶》

0x71 群

给定一个集合 $G=\{a,b,c,\dots\}$ 和集合上的二元运算 "*",如果满足以下条件：

(1) **封闭性**。对于任意的 $a,b\in G,a*b\in G$ 成立。

(2) **结合律**。对于任意 $a,b,c\in G,a*(b*c)=(a*b)*c$ 成立。

(3) **存在单位元**。G 中存在一个元素 e，使得对于 G 中任意元素 a，都有$a*e=e*a=a$，元素 e 为单位元素。

(4) **存在逆元**。对于 G 中任意元素 a，恒有一个 $b\in G$，使得 $a*b=b*a=a$，则元素 b 为元素 a 的逆元，记作 a^{-1}.

则称集合 G 是运算 "$*$" 下的一个群，记为 $(G，*)$ 若 G 是一个有限集，则称 $(G，*)$ 为有限群，其中有限群元素个数成为有限群的阶。记作 $|G|$ ； 若 G 是无限集，则称$(G，*)$ 是无限群。

如果群G中任意两个元素a，b满足交换律，则该群为Abel群。

子群

设 G 在 "$*$" 下是一个群，若 H 是 G 的非空子集且 H 在 "$*$" 运算下也是一个群，则称 $(H，*)$ 是 $(G，*)$ 的子群。

置换

置换，就是一个 1~ n 的排列，是一个 1~ n 排列对 1 ~ n 的映射。其实置换就是给我们一个 n 个数的排列 $1,2,\cdots n$ ，我们通过置换得到 1 ~ n 的映射 $p_1,p_2,p_3\cdots p_n$。

它们一一对应：$\begin{pmatrix}1\ 2\ 3\ ...n\\ p_1\ p_2\ p_3...p_n\end{pmatrix}$

也可以将置换看作是一个**函数** $f(x)$ ，其中 $f(1)=p_1$ ， $f(2)=_2\cdots f(n)=p_n$。

其中 $p_1,p_2,\cdots,p_n$ **仍然是** 1 ~ n 的一个排列，所以很明显不同的置换一共有 $n!$ 种 ， 因为所有排列有一共 $n!$ 种选择方式。

置换的乘法

我们发现置换实际上就表示为**元素位置的变化**。

置换实际上就表示元素的变化，因此就有**置换的乘法**：

$P_1=\begin{pmatrix}1&2&3&4\\ a_1&a_2&a_3&a_4\end{pmatrix}$ $P_2=\begin{pmatrix}a_1\ a_2\ a_3\ a_4\\ b_1\ b_2\ b_3\ b_4\end{pmatrix}$

则 $P_1*P_2=\begin{pmatrix}1&2&3&4\\ b_1&b_2&b_3&b_4\end{pmatrix}$

很明显置换并不支持交换律。

置换的循环与对换

循环是表示置换的一种方法，能反映置换的结构，且便于运算。因此我们把 m 阶循环记为

$\begin{pmatrix}a_1\ a_2...\ a_{m-1}\ a_m\\ a_2\ a_3...\ a_m\ a_1\end{pmatrix}$

特别的，当 $m=2$ 时，2 阶循环 (i,j) 叫做 i 和 j 的对换或换位。（很形象，不解释）

不相交的： 如果两个循环没有公共元素，则称这两个循环是不相交的。
定理13.1： 任何一个置换都可以表示成若干个互不相交的循环的乘积，且表示法是唯一的。

推论13.2： 任何一个置换都可以表示成若干个对换的乘积。因为任何一个循环都可以表示成若干个对换的乘积。

稳定核： 设 是 的子群，若 是 中的某个整数， 中使数 固定不变的置换全体所构成的集合，称为数 在 下的稳定核，或称 中使数 固定不变的置换群，记为 。此时， 为一个不变元。 中不变元的个数记为 。

Example 13.3： 置换 $(a_1\ a_2\ a_3\ a_4\ \cdots a_m)$:

$(a_1\ a_2\ a_3\ a_4\ \cdots a_m)=(a_1\ a_2)(\ a_2\ a_3)(\ a_3\ a_4)\cdots(\ a_d-1\ a_d)$

置换群

一个置换规定为一种变换法则，将集合中的一些元素映射成另一些元素。

一般可以认为 “元素” 是一个数组 $\{a_i\}$，对这个数组的变换（如交换某两个元素，翻转等等）就是置换，置换群就是一个置换的集合加上一个 “叠加” 运算（就是两个置换一次操作）。

在置换群的作用下，元素存在**等价关系**。等价关系即满足**自反性**、**对称性**、**传递性**。满足等价关系的元素处于同一个**等价类**中。

置换群的子集同样是一个置换群。

0x71 Burnside引理与Polya定理

有了上面的这些基础知识，我们就可以引入 Burnside 引理了

Burnside引理

- **不动点**： 不动点 $c(a_i)$ ：若某元素在置换 a_i 下不改变，则成它为置换 a_i 的不动点。
- **元素轨道E_k（等价类）：** 一个元素经过置换能得到的所有元素集合（这里元素可以看做一个点，置换可以看作走一条边，轨道就是能走到的所有点的集合）。
- **稳定化子Z_k：** 使操作后这个元素不变的**置换集合**（即这个元素是此集合内**所有置换的不动点**）。
- **拉格朗日定理：** 一个有限群的子群的元素个数必能整除这个群的元素个数。
- **轨道 - 稳定化子定理：** $|E_k| * |Z_k| = G$

由上式即可推出

定理21.1.1：《组合数学》中的 **Burnside 引理**：

设 G 是 X 的置换群，而 C 是 X 中一个满足下面条件的着色集合：对于 G 中所有的 f 和 C 中所有的 c 都有 $f * c$ 仍在 C 中，则 C 中非等价着色数 $N(G,C)$ 由下式给出：

$$N(G,C)=\frac{1}{|G|}\sum_{f\in G}|C(f)|$$

换言之， C 中非等价的着色数等于在 G 中置换作用下保持不变的着色的平均数。

即：

每个置换的不动点的个数的平均值就是不同的方案数。

0x22 Polya 定理

我们发现如果要使用 Burnside引理 求解计数问题，还需要一个一个数不动点的个数，非常难数，若是数据非常大的话基本上就废了，因此我们引入 Polya 定理，提供了一种直接计算不动点的方法，可以快速方便地解决 Burnside引理 难以求解的问题。

我们上面介绍了任意一个置换都可以分解为循环的乘积。

这里定义 **循环** 的 **循环节** 为循环的个数。

例如：

循环 $\begin{pmatrix}1&2&3&4&5\\3&5&1&4&2\end{pmatrix}=(1\ 3)(2\ 5)(4)$（可以理解为13对应31，25对应52，4对应4）

循环 $(1\ 3)(2\ 5)(4)$ 的循环节为 3 。

对于一个置换 f，$C(f)=k^{m(f)}$，其中 k 为颜色数，$m(f)$ 为 f 的循环节。

综上，在置换群中，**等价类个数（方案数）等于所有置换 f 的 $k^{m(f)}$ 的平均数。**

简单解释一下：

我们假设一个置换有 k 个循环，显然每个循环对应的所有位置颜色需一致，而任意两个循环之间选什么颜色互不影响。因此，若有 k 种可选颜色，则该置换 f 所对应的不动点个数为 $k^{m(f)}$ 。我们用它来替换掉 burnside 引理中的 $C(f)$，即 $C(f)=k^{m(f)}$。得到等价类数量（总方案数）为：

$$N=\frac{\sum_{i=0}^{|G|}k^{m(f_i)}}{|G|}$$

其中 $|G|$ 表示置换的数目，$m(f_i)$ 表示第 i 个置换包含的循环个数。

Problem 22.1 ： 【经典例题 1 】

如图 22.1.1 所示，2×2 方格中每个格子可以选择染上 2 种颜色（红色或白色）。那么总共是 $2^4=16$ 种情况。现在要问，如果 不旋转、顺时针旋转 90 度、逆时针旋转 90 度、旋转 180 度后相同的均算成同一种方案，问总共有多少种不同的方案。

Solution

我们根据 Polya 定理可以得到下式：

不旋转

$$f_1=\begin{pmatrix}1&2&3&4\\1&2&3&4\end{pmatrix}=(1)(2)(3)(4),C(f)=k^{m(f)}=2^4=16$$

顺时针旋转 90°

$$f_2=\begin{pmatrix}1&2&3&4\\2&3&4&1\end{pmatrix}=(1\ 2\ 3\ 4),C(f)=k^{m(f)}=2^1=2$$

逆时针旋转 90°

$$f_3=\begin{pmatrix}1&2&3&4\\4&1&2&3\end{pmatrix}=(1\ 2\ 3\ 4),C(f)=k^{m(f)}=2^1=2$$

旋转 180°

$$f_4=\begin{pmatrix}1&2&3&4\\3&4&1&2\end{pmatrix}=(1\ 3)(2\ 4),C(f)=k^{m(f)}=2^2=4$$

这里**旋转** 180° 得到的循环的乘积为 $(1\ 3)(2\ 4)$ ，即代表 1、3 循环置换，互相变换，2、4 循环置换，很明显 1 和 3 的颜色必须相同，2 和 4 的颜色也必须相同，而 1、3 和 2、4 的颜色互不相干，没有影响。

最后我们使用 **Polya 定理** 同样可以得到正确答案：

$$N=\frac{16+2+2+4}{4}=6$$

0x23 特殊模型

0x23.1 旋转同构

模型简述： n个点，每个点移动 k 步（$0\le k\le n-1$），循环个数为：$\gcd(k,n)$

证明：

1. 当 k 是 n 的约数，显然成立。一个环用 $\frac{n}{k}$ 个，可以分成$\frac{N}{(\frac{N}{k})}=k$ 个循环（$gcd(k,n)=k$）。
2. 当 k 不是 N 的约数，最小的循环长度是：$\mathrm{lcm}(k,n)$，循环使用的端点是：$\frac{\mathrm{lcm}}{k}$个，可以凑成 $\frac{n}{\frac{lcm}{k}}=\frac{n\times k}{\mathrm{lcm}}=gcd(N,k)$ 个。

Problem 23.1.1：循环同构

我们发现经典问题实际上就相当于：圆上有 4 个点，2 种颜色，旋转同构，求方案数。 我们来把它扩展一下：圆上有 n 个点，k 种颜色，旋转同构，求总方案数。

Solution

仔细分析可以发现，当我们旋转 i 个点时，一共会有 $\gcd(i,n)$ 个循环，每个循环会有 $\frac{n}{\gcd(i,n)}$ 个元素。

因此
$ans=\frac{1}{n}\sum_{i=0}^{n-1}k^{\gcd(i,n)}$

0x23.2 对称同构

Problem 23.2.1：对称同构

圆上有 n 个点，k 种颜色，旋转同构，翻转同构，求总方案数。

Solution

旋转同构总共形成 $a=\sum_{i=0}^{n-1}k^{\gcd(i,n)}$ 个不动点。

翻转同构分两种情况考虑：

当 n 为奇数时，对称轴有 n 条，每条对称轴形成 $\frac{n-1}{2}$ 个长度为 2 的循环，1 个长度为 1 的循环（对称轴一定过一个顶点），共 $\frac{n-1}{2}+1=\frac{n+1}{2}$ 个循环。

因此总共形成 $b=nk^{\frac{n+1}{2}}$ 个不动点。

当 n 为偶数时，有两种对称轴。穿过两个点的对称轴有 $\frac{n}{2}$ ，共形成 $\frac{n}{2}+1$ 个循环；不穿过点的对称轴有 $\frac{n}{2}$ ，共形成 $\frac{n}{2}$ 个循环。

因此总共形成 $b=\frac{n}{2}\left(k^{\frac{n}{2}+1}+k^{\frac{n}{2}}\right)$ 个不动点。

综上，$ans=\frac{a+b}{2n}$

奇数点按边对称： $\frac{n-1}{2}+1=\frac{n+1}{2}$ 个循环。

奇数点按边对称： $\frac{n}{2}$ 个循环。

按点对称： $2+\frac{n-2}{2}=\frac{n+2}{2}$ 个循环。

0x80 递推方程

线性递推方程

$F_n-b_1\times F_{n-1}-b_2\times F_n-2-\ldots-b_k\times F_n-k=0$
其通项公式为：$F_n=c_1\times q_1^n+c_2\times q_2^n+\ldots+c_k\times q_k^n$
其中 $q_1\ldots q_n$ 是特征方程，$q^k-b_1\times q^(k-1)-b_2\times q^(k-2)-\ldots-b_k=0$ 的根
$c_1\ldots c_k$ 是常数，由初值决定

非线性递推方程：

$F_n-b_1\times F_{n-1}-b_2\times F_{n-2}-\ldots-b_k\times F_n-k=S(n)$
其通项公式为：$F_n=c_1\times q_1^n+c_2\times q_2^n+\ldots+c_k\times q_k^n+f_n$
其中 $q_1\ldots q_n$ 是特征方程的根，f_n为一特解

0x90 康托展开

康托展开是一个全排列到一个自然数的双射，常用于构建哈希表时的空间压缩，在组合数学中，其解决的是当前排列在全排列中的名次问题。

简单来说，给定一个 n 位数的全排列，可根据康托展开公式确定其应是字典序中的第几个排列。

由于康托展开计算的是某个全排列方式在该全排列集合中的字典序，其映射关系是唯一的，而且单调，因此映射关系是可逆的，故而当给定一个全排列的所有字符，以及某个字典序编号，可以利用逆康托展开得到相应的那个全排列。

0x91 康托展开

对于康托展开，有公式：$X=a[n]*(n-1)!+a[n-1]*(n-2)!+a[n-2]*(n-3)!+\ldots+a[1]*0!$，其中 a[i] 表示原排列中，排在下标 i 后的，比下标 i 的字符还小的字符个数

通过公式算出来的康托展开值，是指当前序列在之前的全排列的个数，因此 X+1 即为该序列在全排列中的次序。

以序列 {3,2,5,4,1} 为例：

对于 3：比 3 小的有 1、2，所以 3 是第 2 小的，$X+=2*(5-1)!$
对于 2：比 2 小的有 1，所以 2 是第 1 小的，$X+=1*(4-1)!$
对于 5：比 5 小的有 1、2、3、4，但由于 2、3 已经出现过了，所以目前 5 是第 2 小的，$X+=2*(3-1)!$
对于 4：比 4 小的只剩 1，所以 $X+=1*(2-1)!$
对于 1：已经是最小的，$X+=0*(1-1)!$
因此，
$X=2*(5-1)!+1*(4-1)!+2*(3-1)!+1*(2-1)!+0*(1-1)!+1=48+6+4+1+0+1=60$

因此，序列 {3,2,5,4,1} 在由数 1~5 组成的全排列中的次序是 60

```
int a[N];
int fac[N];
void getFactor(int n) { //计算阶乘
    fac[0] = 1;
    for (int i = 1; i ≤ n; i++)
        fac[i] = fac[i - 1] * i;
}
int contor(int n) { //康托展开
    int X = 0;
    for (int i = 1; i < n; i++) {
        int cnt = 0;
        for (int j = i + 1; j ≤ n; j++)
            if (a[j] < a[i])
                cnt++;
        X += cnt * fac[n - i];
    }
    return X + 1;
}
int main() {
    int n;
    scanf("%d", &n);
    getFactor(n);

    for (int i = 1; i ≤ n; i++)
        scanf("%d", &a[i]);

    int res = contor(n);
    printf("%d\n", res);
    return 0;
}
```

0x92 逆康托展开

康拖展开是从序列到自然数的映射且是可逆的，那么逆康拖展开便是从自然数到序列的映射，简单来说，逆康托展开，就是给出 1~n 的数列，求出排名第 x 的数

以序列 $\{1,2,3,4,5\}$ 为例，求第 10 的序列：

有：$X=10$，$X-1=9$，那么：

第一个数：9/(5-1)!=0...9，说明比第一个数小的、没有出现过的数不存在，因此第一个数是：1
第二个数：9/(4-1)!=1...3，说明比第二个数小的、没有出现过的数有 1 个，因此第二个数是：3
第三个数：3/(3-1)!=1...1，说明比第三个数小的、没有出现过的数有 1 个，因此第三个数是：4
第四个数：1/(2-1)!=1...0，说明比第四个数小的、没有出现过的数有 1 个，因此第四个数是：5
第五个数：0/(1-1)!=0...0，说明比第五个数小的、没有出现过的数不存在，因此第五个数是：2
因此，第 10 的序列为 {1,3,4,5,2}

```cpp
int fac[N];
bool vis[N];
void getFactor(int n) { //计算阶乘
    fac[0] = 1;
    for (int i = 1; i ≤ n; i++)
        fac[i] = fac[i - 1] * i;
}
vector<int> revContor(int n, int X) {//逆康托展开
    memset(vis, false, sizeof(vis));
    vector<int> res(n, -1);
    X--;
    int residue = X; //除数
    for (int i = 0; i ≤ n - 1; i++) {
        int cnt = residue / (fac[n - i - 1]);
        residue = residue % (fac[n - i - 1]);
        for (int j = 1; j ≤ n; j++) {
            if (!vis[j]) {
                if (!cnt) {
                    vis[j] = true;
                    res[i] = j;
                    break;
                }
                cnt--;
            }
        }
    }
    return res;

}
int main() {
    int n, num;
    scanf("%d%d", &n, &num);
    getFactor(n);
    vector<int> res = revContor(n, num);

    for (int i = 0; i < res.size(); i++)
        cout << res[i];

    return 0;
}
```

参考资料：

- 组合数学学习笔记
- 《组合数学》读书笔记
- ACM-组合数学完全总结（知识点+模板）
- 排列组合的一些公式及推导(非常详细易懂)
- 组合数学笔记
- 浅谈生成函数与组合计数，多项式全家桶（未完）
- 组合计数 学习笔记
- 组合数学总结